

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY :

Software Engineering

MES-X Dependency Management

Traceability and Release-Readiness Analysis on Azure DevOps

Prepared by

Alaa Eddine Mdalla

Internship carried out at

Forvia Informatique Tunisie

Academic Year

2024/2025

Acknowledgments

This work would not have been possible without the support, guidance, and kindness of the many people who accompanied me throughout this journey.

First and foremost, I would like to express my sincere gratitude to **Ms. Khemiri Karima**, **Abassi Chifa** and **Ms. Gabsi Oumeima** for their constant support, thoughtful guidance, and genuine encouragement throughout my internship at Forvia. Their professionalism, trust, and kindness made this experience both enriching and memorable, and I remain deeply thankful for the confidence they placed in me from the very beginning.

I am equally grateful to **Ms. Boumaiza Wafa** for her attentive academic supervision and the valuable advice she provided at every stage of this work. Her patience, rigor, and continuous encouragement were a constant source of motivation and played an essential role in shaping this project.

I would also like to extend my heartfelt thanks to the **MES-X team** for their collaboration, openness, and the welcoming environment they created throughout this experience. I am especially grateful to **GHAZOUANI Mohamed Neji**, **BEJAOUI Nawres**, **SALAH Houssam**, and **YOUAZ Ichrak**, whose support, availability, and team spirit contributed greatly to both my professional growth and the success of this internship.

Beyond the professional sphere, I am profoundly grateful to the people who have been my pillars throughout this journey.

My first and most heartfelt thought goes to my **late father, Sammi Mdalla**, who passed away last March. He did not have the chance to see this work completed, but his presence lives on in every page of it. His sacrifices, his values, and his unwavering belief in me are the foundation upon which everything I have achieved has been built. I carry his memory with me always.

To my **mother, Hedia Ben Jouda**, thank you for your boundless love, your patience, and your strength. You have always been my refuge and my source of courage. To my **brother, Dhia Mdalla**, and my **sister, Wafa Mdalla**, thank you for your affection, your presence, and for always believing in me, even in moments when I doubted myself.

This accomplishment belongs to all of you.

Abstract

Large software teams working with Azure DevOps need a reliable way to trace the links between work items, pull requests, commits, branches, and release tags. Within the MES X.0 environment at Forvia, this traceability was previously built through a manual process that was slow, repetitive, and error-prone.

The **MES-X Dependency Management** project addresses this limitation by providing a read-only traceability service composed of a NestJS backend and a Vue 3 frontend. The backend integrates with Azure DevOps, traverses dependency graphs, resolves pull requests and commits, streams batch results, and identifies the tag context of each change. The frontend gives users an operational interface for launching analyses, following progress, and reviewing consolidated results.

This report presents the project context, the formal requirements, the proposed architecture, the implementation choices made for both backend and frontend, and the main technical outcomes of the internship.

Keywords: NestJS, Azure DevOps, Vue 3, dependency management, traceability, internship report.

List of Figures

1.1	Azure DevOps environment used within the MES X.0 division, showing work item tracking and development workflow management	4
3.1	Use Case Diagram of the MES-X Dependency Management System. . .	24
4.1	Global three-layer architecture of the proposed solution.	28
4.2	Sequence diagram — Work Item Management Component.	34
4.3	Sequence diagram — Pull Request Resolution Component.	36
4.4	Sequence diagram — Commit Aggregation Component.	38
4.5	Sequence diagram — Decision Aggregation Component.	40
4.6	End-to-end data flow across the four system components.	43
5.1	Frontend technology used: Vue 3 for building the user interface	51
5.2	Backend technology used: NestJS for scalable server-side architecture .	52
5.3	Azure DevOps Platform	53
5.4	Agile Scrum methodology applied during system development	53
5.5	Backend module-level class diagram	57
5.6	Sequence diagram of single-root related-work-item processing	59
5.7	Sequence diagram of batch related-work-item aggregation	60
5.8	UI route/component architecture for traceability workflows	61
5.9	Sequence diagram of UI and backend integration in trace workflows . .	62
5.10	Batching Work Items — selection and SSE-driven progress	63
5.11	Related Work Item — single-root traceability graph view	63
5.12	Discovered Commits — commits discovered during traversal	64

5.13	Decision — work items grouped by state (Done vs Other)	64
5.14	Decision — repository view with expected tags for promotion	65
5.15	Backend Docker orchestration and runtime communication model . . .	66

List of Tables

3.1	Functional Requirements to System Component Traceability Matrix . .	25
-----	---	----

List of Acronyms

Abbreviation	Definition
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
DTO	Data Transfer Object
DTOs	Data Transfer Objects
E2E	End-to-End
Epic	High-level Azure DevOps work item used to group major business capabilities
FR	Functional Requirement
Git	Distributed version control system used by Azure DevOps repositories
MES	Manufacturing Execution System
MES-X	Forvia's Manufacturing Execution System platform context for this dependency-management service
NFR	Non-Functional Requirement
NestJS	Node.js framework used to build the backend
PAT	Personal Access Token
PR	Pull Request

Abbreviation	Definition
Quasar	Vue-based UI framework used to build the frontend interface
REST	Representational State Transfer
SSE	Server-Sent Events
TypeScript	Typed superset of JavaScript used in both backend and frontend development
UI	User Interface
UML	Unified Modeling Language
UC	Use Case
Vite	Frontend build tool used to bundle and serve the Vue application
Vue	Progressive JavaScript framework used for the frontend application

General Introduction

Managing a large software platform is never only a matter of writing code. In the MES X.0 environment at Forvia Informatique Tunisie, teams work on a microservices-based Manufacturing Execution System deployed across several industrial sites. Azure DevOps supports this work by centralizing planning, source control, code review, and pipeline automation.

The real difficulty lies in understanding how all of these development artifacts relate to one another. A single feature may involve several work items, multiple pull requests, and many commits spread across different repositories. Before this project, tracing those links required a largely manual process based on Azure DevOps navigation and spreadsheet tracking. The approach was slow, repetitive, and difficult to reproduce reliably.

This internship addressed that problem by designing and implementing an automated traceability system able to reconstruct dependency chains directly from Azure DevOps data. The objective was not simply to save time, but to give developers and release engineers a clearer and more dependable view of the relationships between requirements, code changes, and release tags.

The rest of this report follows the progression of that work. It first presents the industrial context and the project objectives, then formalizes the system requirements, explains the proposed architecture, and finally describes the implementation choices and the main results achieved during the internship.

Chapter 1

Project Overview

1.1 Introduction

This chapter establishes the professional and technical context of the end-of-study internship project. It presents the host company, the industrial environment in which the work was carried out, the problem that motivated the project, and the main objectives assigned during the internship.

This context is important because the technical decisions described in the next chapters were shaped by concrete industrial constraints: a large microservices landscape, a growing number of Azure DevOps artifacts, and the operational need for faster and more reliable traceability.

1.2 Host Company

1.2.1 The Forvia Group

Forvia is one of the world's leading automotive technology groups, formed through the strategic merger of two major European industrial players: **Faurecia**, a global specialist in automotive seating, interiors, and emissions control technologies, and **HELLA**, a globally recognized leader in automotive lighting systems and electronic components. The merger created a group of exceptional scale, combining complementary technological portfolios and international footprints to serve automotive manufacturers across every major global market.

Forvia operates across more than 40 countries, employs over 150,000 people worldwide, and maintains a strong research and development focus aimed at accelerating

the transition toward sustainable, intelligent, and connected vehicles. Its product and technology portfolio spans seating systems, cockpit electronics, clean mobility solutions, lighting, and sensors — placing the group at the intersection of several of the most critical transformation vectors in the modern automotive industry.

1.2.2 Forvia Informatique Tunisie

The internship took place at **Forvia Informatique Tunisie**, the Tunisian IT and digital engineering hub of the Forvia Group. Established as a center of technical excellence within the group’s global digital organization, this entity is responsible for the design, development, and long-term maintenance of the software systems that support industrial operations at Forvia’s manufacturing facilities worldwide.

Forvia Informatique Tunisie concentrates a multidisciplinary team of software engineers, architects, DevOps specialists, and technical leads who work in close collaboration with operational and product teams across multiple Forvia business units. The entity contributes to a broad range of digital initiatives, including manufacturing execution systems, production monitoring platforms, enterprise integration solutions, and internal tooling designed to improve the efficiency and traceability of the group’s software delivery processes.

1.2.3 The MES X.0 Division

The project was carried out within the **MES X.0** division of Forvia Informatique Tunisie. This division is responsible for the design, development, and evolution of a next-generation Manufacturing Execution System — a class of industrial software that serves as the operational backbone of manufacturing plants by orchestrating, monitoring, and recording all production activities in real time.

The MES X.0 platform is built on a **microservices architecture**, a deliberate technical choice that enables independent development, deployment, and scaling of each functional domain of the system. Rather than a single monolithic application, the platform is composed of dozens of specialized services — each responsible for a specific manufacturing concern such as production scheduling, quality control, equipment monitoring, or operator guidance — that communicate through well-defined APIs and messaging contracts.

This architecture gives the platform exceptional flexibility and scalability: new manufacturing capabilities can be added as new services, existing services can be updated independently without impacting the whole system, and different plants can

be equipped with different subsets of the platform’s services according to their specific operational needs. The MES X.0 platform is deployed and actively used at multiple Forvia manufacturing sites around the world, processing real-time production data from industrial equipment and human operators simultaneously.

However, this architectural flexibility comes with a corresponding increase in the complexity of the software development process — a complexity that directly motivated the project described in this report.

1.3 Project Context and Problem Statement

1.3.1 Development Environment at Forvia

At Forvia, **Azure DevOps** serves as the central platform for the software development lifecycle across all digital teams. It provides an integrated suite of tools covering the full range of development activities: sprint and release planning through work item boards, source code management through Git repositories, code review and integration through pull requests, automated build and deployment pipelines through Azure Pipelines, and artifact management for binaries and package versions.

Within the MES X.0 division specifically, Azure DevOps manages the development activities of all microservices simultaneously. A given sprint may involve planned work across ten, twenty, or more independent services, each progressing at its own pace with its own team, its own repository, and its own release cadence. The platform accumulates an ever-growing set of interconnected artifacts: work items describing requirements and tasks, pull requests integrating code changes, commits recording individual code modifications, branches isolating development streams, and repository tags marking release points.

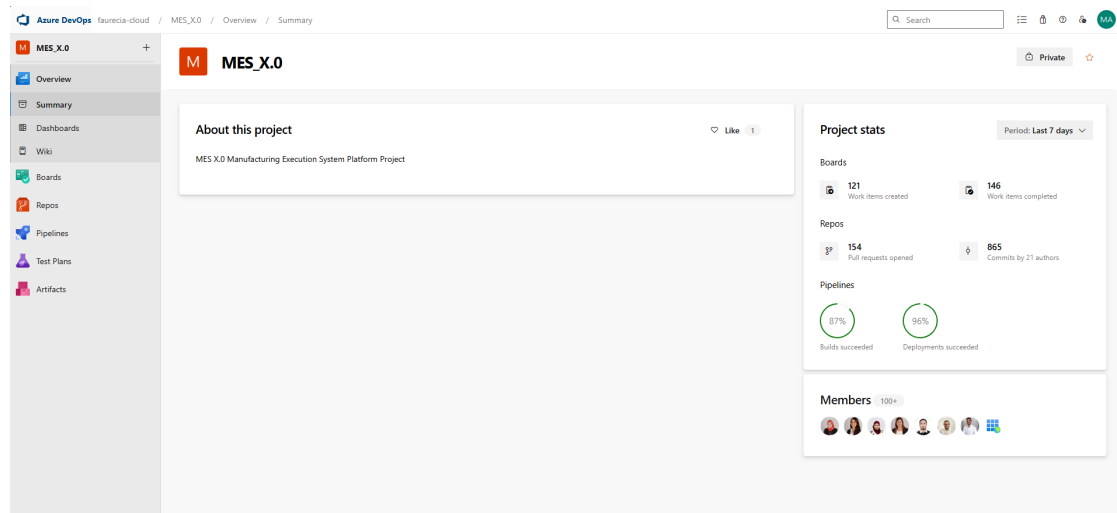


Figure 1.1. Azure DevOps environment used within the MES X.0 division, showing work item tracking and development workflow management

1.3.2 The Traceability Challenge

In this multi-service, high-velocity development environment, a single user-facing requirement — expressed as a work item at the Epic or Feature level in Azure DevOps — typically decomposes into dozens of lower-level tasks, each of which produces its own chain of development artifacts: a branch, one or more commits, a pull request, and eventually a repository tag when the change is released.

Understanding the complete impact of a requirement — or conversely, understanding what requirements are affected by a change in a specific repository — requires navigating this artifact chain in its entirety. For a release engineer preparing a production deployment, this means answering questions such as:

- Which work items are included in this release, and are they all in a completed state?
- For each work item, which pull requests carried the implementation, and in which repositories?
- For each repository, which commits were produced, and do they appear before or after the relevant release tag?
- Are there any commits that straddle a release boundary — present in one environment but not yet in another?

Before this project, answering these questions required a **manual and entirely ad-hoc process**. The release engineer or developer would begin from a work item in

Azure DevOps, navigate to its linked pull requests, open each PR to identify the target repository, navigate to the repository's commit history, search for the relevant commits, check branch and tag references, and then repeat the entire sequence for every related work item. Data extracted at each step was typically recorded manually in spreadsheets to maintain a working reference.

This process presented several critical limitations:

- **Time cost:** a complete dependency trace for a single feature involving multiple services could require an entire working day to complete manually.
- **Error proneness:** the manual correlation of data across multiple Azure DevOps interfaces introduced a significant risk of missed links, incorrect associations, and incomplete coverage.
- **Non-reproducibility:** two engineers performing the same trace independently could arrive at different results depending on the path they followed and the data they chose to record.
- **Scalability failure:** as the number of microservices grew and development velocity increased, the manual approach became increasingly impractical, consuming disproportionate engineering time during critical release preparation phases.

1.3.3 Project Motivation

These limitations created a clear and urgent need for an automated solution. The **MES-X Dependency Management** project was initiated within the MES X.0 division to address this need directly: to design and implement a system that would automate the reconstruction of dependency chains across Azure DevOps artifacts, transforming a time-consuming and error-prone manual process into a fast, reliable, and reproducible analytical capability.

The project was scoped as an end-of-study internship project, representing an opportunity to deliver genuine industrial value while applying and extending the software engineering knowledge acquired during the academic program.

1.4 Project Description

1.4.1 Project Objectives

The primary goal of the MES-X Dependency Management project is to design and implement an **automated dependency traceability system** that replaces the existing manual process with a reliable, efficient, and scalable solution integrated into the Azure DevOps ecosystem.

The project pursues the following specific objectives:

Automated Microservice Tag Identification

The system must automatically identify and associate the relevant microservice repository tags with work items by analyzing commit history and repository metadata. This capability eliminates the need for manual navigation through repository tag lists and commit histories, and ensures that the release context of each code change is determined programmatically and consistently.

Comprehensive Dependency Mapping

The system must construct and expose complete traceability links across the full chain of development artifacts: from high-level work items through pull requests and commits down to repository branches and tags. This mapping must be complete — covering all levels of the work item hierarchy and all artifact types — and consistent across successive analysis sessions.

Significant Reduction of Manual Effort

By automating data retrieval, traversal, normalization, and aggregation, the system must reduce the time required to perform a complete dependency analysis from hours to seconds. This reduction must be sustained even for complex analysis scenarios involving multiple work items and many repositories.

Improved Data Accuracy and Consistency

The system must eliminate the class of errors introduced by manual data extraction and correlation. All dependency data must be derived from the same authoritative source —

the Azure DevOps APIs — through a standardized and reproducible process, ensuring that results are accurate and identical across successive runs for the same input.

End-to-End Traceability

The system must enable a user to trace any work item from its initial description to the exact commits and microservice tags it affected, entirely within a single analysis session. No cross-tool navigation or manual correlation should be required to obtain a complete dependency view.

Reusable and Scalable Architecture

The system must be built on a modular, maintainable architecture that can be extended to support additional artifact types, additional analysis modes, or integration into other Forvia digital platforms — without requiring a fundamental redesign of the existing system.

1.4.2 Technical Scope

The project encompasses the design and implementation of three interconnected technical components: a backend service, a frontend application, and an Azure DevOps integration layer.

Backend Development

The backend was developed using **NestJS** with TypeScript, a framework chosen for its native support of modular architecture, dependency injection, and TypeScript-first development — properties well-suited to the clean, maintainable design required by this project.

The backend’s responsibilities include:

- Secure integration with the Azure DevOps REST APIs using Personal Access Token authentication.
- Normalization of raw Azure DevOps responses into structured, typed Data Transfer Objects that form the stable internal data model of the system.
- Recursive traversal of work item dependency graphs across multiple hierarchy levels and relation types.

- Resolution of pull requests, commits, repository branches, and Git tags for each node in the dependency graph.
- Implementation of batch processing strategies to minimize API call volume and respect rate limits.
- Implementation of Server-Sent Events streaming for progressive delivery of large analysis results.
- Exposure of well-documented RESTful endpoints, complemented by interactive Swagger documentation for developer exploration and testing.

Frontend Application

The frontend was developed using **Vue 3** with the Quasar framework, providing a production-grade, responsive interface for initiating and exploring dependency analyses. The frontend's role is not limited to presentation: it actively participates in the orchestration of analysis sessions by managing asynchronous backend calls, assembling view state from streamed results, and providing real-time progress feedback to the user.

Key features of the frontend include:

- Interactive visualization of work item dependency graphs, presenting hierarchical relationships, pull request links, and commit references in a structured and navigable form.
- Real-time batch processing interface powered by Server-Sent Events, allowing users to observe analysis progress incrementally rather than waiting for a single blocking response.
- Clear presentation of relationships between work items, pull requests, commits, branches, and tags, organized by repository and analysis mode.
- A dedicated Decision Dashboard that presents the release classification output of the aggregation component in an actionable, decision-ready format.
- A responsive and operationally focused design, tailored for everyday use by release engineers and development team leads.

Azure DevOps Integration and Data Processing

The solution relies on the Azure DevOps REST API as its exclusive data source. The integration layer is responsible for all communication with this external system and implements several mechanisms to ensure reliability and efficiency:

- Authentication via Personal Access Tokens with secure credential management.
- Intelligent batching and concurrency control to respect Azure DevOps API rate limits during high-volume analysis sessions.
- A robust dependency resolution engine capable of recursive graph traversal, entity deduplication, and cross-module result normalization.
- Structured error handling and retry logic for transient API failures, ensuring that partial external failures do not silently corrupt analysis results.

1.4.3 Project Activities

The project followed a structured development lifecycle organized around the following phases:

- **Requirements Analysis:** study of the existing manual traceability workflow, interviews with release engineers and developers, and formalization of the system's functional and non-functional requirements.
- **System Design and Architecture:** definition of the three-layer architecture, identification of the four main backend components, design of API contracts and data transfer objects, and preparation of the frontend structure.
- **Backend Implementation:** development of the Azure DevOps integration layer, recursive traversal logic, pull request resolution, commit and tag processing services, and progressive streaming support.
- **Frontend Development:** implementation of the main analysis views, centralized state management with Pinia, integration of real-time SSE updates, and development of the Decision Dashboard.
- **Testing and Validation:** unit testing of backend services and utilities, validation of backend contracts against frontend needs, and end-to-end checks for the main analysis scenarios.
- **Integration and Optimization:** performance tuning of graph traversal and batch resolution, implementation of in-memory caching, and verification of result consistency across analysis modes.
- **Deployment Preparation:** backend containerization with Docker, environment configuration, CI/CD pipeline setup, and completion of Swagger documentation for developer handoff.

1.5 Conclusion

This chapter has presented the industrial setting of the MES-X Dependency Management project and clarified why dependency tracing had become a real operational issue. In a large microservices ecosystem, manual analysis no longer provides the speed, consistency, or reliability required for release preparation.

The next chapter translates that problem into a formal requirements specification. It defines what the system must deliver functionally and which quality constraints it must satisfy in practice.

Chapter 2

Study of the Existing

2.1 How Dependency Tracking Is Done Today

This chapter examines the manual process used before the introduction of the MES-X Dependency Management system. Understanding that workflow is important, because the design choices made later in the report are direct responses to its limitations.

In the MES X.0 environment, the starting point is usually a **user story** in Azure DevOps. That story is often linked to several child tasks, and each task may produce its own pull request, commits, and release references in different repositories. Azure DevOps stores all of these artifacts, but it does not provide a single consolidated view of their dependency chain.

In practice, a developer preparing a dependency report had to repeat the same sequence for *each* user story under analysis:

1. Open the work item in Azure DevOps and note all linked child tasks and related items.
2. Navigate to the **Development** panel of each item to find associated pull requests.
3. Open every pull request individually, then switch to its **Commits** tab.
4. Inspect each commit message and the list of changed files to identify which microservice is affected and extract any version tags present in the commit.
5. Record the collected information—work item ID, PR number, commit hash, microservice name, tag—row by row in an Excel spreadsheet.
6. Repeat the entire sequence for every child work item linked to the original story.

During a typical sprint, with ten to fifteen user stories and several pull requests per story, this work could easily occupy most of a day. Every link had to be opened manually, and every relevant tag had to be checked one by one.

2.2 Illustrative Example

Consider a user story such as “*Update shared configuration service for plant B.*” The change may involve the configuration service itself, a dependent API gateway, and a deployment adjustment. In practice, that single story can be split into several child tasks, each with its own pull request and its own set of commits.

To build a dependency view for that story, the developer must inspect each pull request, identify the affected repository, review the associated commits, and locate the relevant release tags. Even in a modest case, this means opening several pull requests and reading through dozens of commits. Once this effort is multiplied across a full sprint, the workload becomes substantial and the risk of omission increases.

2.3 Limitations and Their Consequences

The manual process creates four major problems.

Time consumption. Preparing a complete dependency report for a sprint can take from several hours to a full working day. That time is repeatedly spent on navigation and transcription instead of implementation or review.

High error risk. The result depends on careful human inspection. A single missed commit, an overlooked repository, or an incorrect tag reference can produce an incomplete report and lead to poor release decisions.

Poor scalability. The MES X.0 platform already contains many microservices, and its scope continues to grow. As more repositories are added, the manual approach becomes less realistic and more costly.

Lack of a shared source of truth. Because the final result is often stored in spreadsheets, two engineers can produce different analyses for the same feature with no automatic way to reconcile the difference.

These limitations affect both productivity and reliability. They justify the need for a structured, automated solution, which is formalized in the next chapter through the system requirements.

Chapter 3

Requirements Specification

3.1 Introduction

This chapter defines the requirements of the MES-X Dependency Management system. At this stage, the objective is to describe the expected behavior of the solution independently from implementation details, so that the later architecture and implementation can be evaluated against a clear baseline.

The requirements were derived from the manual workflow described in the previous chapter, discussions with developers and release engineers, and direct observation of the traceability tasks performed in the MES X.0 context. They cover both the services the system must provide and the quality constraints it must respect in daily use.

3.2 Functional Requirements

Functional requirements describe the behavior the system must provide from the user and process perspective. They define the capabilities needed to trace artifacts across work items, pull requests, commits, and tags.

3.3 Work Item Traceability Requirements

Work items are the usual starting point for traceability analysis. They express the needs, tasks, and user stories that define the expected behavior of the software, and they serve as the roots for the dependency graph.

- **FR-WI-01 – Single Work Item Retrieval:** The system must be able to

retrieve a work item from Azure DevOps by its numeric identifier and return a normalized view of its metadata, including title, type, state, assignee, and all relation links.

- **FR-WI-02 – Relation Type Interpretation:** The system must be able to interpret the relation types used by Azure DevOps work items, including parent-child links, peer links, and artifact links to external objects such as pull requests. Each relation type must be handled correctly during graph construction.
- **FR-WI-03 – Dependency Graph Traversal:** The system must be able to explore the full dependency graph reachable from a root work item by following relation links across all hierarchy levels. The traversal must be cycle-safe and must support graphs of arbitrary depth without affecting correctness.
- **FR-WI-04 – Multi-Root Batch Analysis:** The system must be able to accept several work item identifiers as the starting points of one analysis session, traverse the graph for each root, and return one aggregated and deduplicated result covering all reachable work items.
- **FR-WI-05 – Pull Request Reference Extraction:** While traversing work item relations, the system must identify and extract all references to associated pull requests, producing a deduplicated list of pull request identifiers for later resolution.
- **FR-WI-06 – Progressive Streaming for Large Datasets:** For analysis sessions involving many work items, the system must support a progressive delivery mode in which partial results are streamed to the client as they are discovered, so the user can follow progress in real time instead of waiting for the full traversal to finish.

3.4 Pull Request Traceability Requirements

Pull requests form the bridge between work tracking and source control. They are the point where implementation work becomes a formal change set in Azure DevOps, so the system must resolve them accurately and enrich them with useful context.

- **FR-PR-01 – Pull Request Metadata Resolution:** The system must be able to retrieve the full metadata of a pull request from the Azure DevOps Git API using its identifier, including title, status, source and target branches, and the repository identifier.

- **FR-PR-02 – Linked Work Item Retrieval:** The system must be able to retrieve the work items formally linked to a pull request on the Git side of Azure DevOps. This list may differ from the work items linked from the work-tracking side, and both views must be supported.
- **FR-PR-03 – Repository Identity Propagation:** For each resolved pull request, the system must extract and propagate the repository name so that downstream components can group and classify results without making additional API calls.
- **FR-PR-04 – Traceability Chain Bridging:** The system must use resolved pull request information to extend the traceability chain from work items into source control. PR resolution must produce the structured context needed for commit and tag resolution.

3.5 Commit Traceability Requirements

Commits are the most detailed part of the traceability chain. They capture the actual code changes behind a requirement and provide the context needed to relate those changes to branches and release tags.

- **FR-CO-01 – Commit Resolution via Pull Request:** The system must be able to retrieve the commits merged as part of a specific pull request using the Azure DevOps Git API. This is the main path from work items to code changes when a formal PR flow was used.
- **FR-CO-02 – Commit Resolution via Work Item:** The system must be able to retrieve commits directly linked to a work item through Azure DevOps relation metadata, independently of any pull request.
- **FR-CO-03 – Commit Deduplication:** When the same commit is reachable through multiple paths, such as a direct work item link and a pull request link, the system must keep only one instance in the final result. Deduplication must happen by commit identifier before downstream processing.
- **FR-CO-04 – Branch Context Resolution:** For a given repository and commit reference, the system must be able to retrieve the branch references that contain the commit, so the development stream can be identified.
- **FR-CO-05 – Tag Context Resolution:** For a given repository and commit reference, the system must be able to identify the repository tags associated with

the commit, specifically the most recent tag reachable from the commit on the relevant branch. This tag context is the main input for release classification.

- **FR-CO-06 – Batch Tag Resolution:** The system must support a batch mode for tag resolution in which the tag context of several commits in the same repository is resolved in a single API interaction. This is necessary to keep performance acceptable on large graphs.

3.6 Data Aggregation Requirements

Aggregation requirements govern how the system consolidates the data collected across the traceability pipeline. The goal is to produce one coherent result that is normalized, deduplicated, and suitable for both automated use and human review.

- **FR-AG-01 – Cross-Domain Result Consolidation:** The system must consolidate the normalized results from work item traversal, pull request resolution, commit resolution, and tag enrichment into one unified data structure representing the complete dependency view.
- **FR-AG-02 – Global Deduplication:** During consolidation, the system must remove duplicate work items, pull request references, and commit identifiers that arise from multi-path traversal. Each entity must appear only once in the final result.
- **FR-AG-03 – Referential Consistency:** The aggregated result must remain internally consistent. Every pull request reference must correspond to a resolved PR record, every commit reference must correspond to a commit record, and every repository name must be used consistently across the result.
- **FR-AG-04 – Decision Classification:** The system must apply a classification rule to the aggregated trace data that groups work items by completion state and repositories into release-relative decision buckets based on commit tag context. The output must be ready for release-readiness assessment without additional manual interpretation.
- **FR-AG-05 – Structured Output Format:** All aggregated outputs must be returned as typed, structured responses that follow documented data contracts. The output structure must be stable, versioned, and independent of internal implementation details.

3.7 Analysis and Reporting Requirements

The system is not only a retrieval tool. It must also turn traceability data into something that helps people make decisions about scope, integration, and release readiness.

- **FR-AN-01 – Dependency Relationship Identification:** The system must explicitly expose the relationships between development artifacts at every level of the chain: which work items are related to which other work items, which pull requests are associated with which work items, and which commits are associated with which pull requests and repositories.
- **FR-AN-02 – Repository-Based Grouping:** The system must be able to group resolved commits and pull requests by repository, producing a per-repository view of the changes associated with the analyzed work items. This is important for release engineers who work service by service.
- **FR-AN-03 – Release Context Interpretation:** The system must interpret the tag context of each commit to determine its position relative to a release boundary, classifying it as preceding, following, or straddling a specified release tag. The interpretation must be applied consistently across all repositories.
- **FR-AN-04 – Multi-Level Traceability View:** The system must produce a complete traceability view across all relevant abstraction levels: epics and features, user stories and tasks, pull requests, commits, and repository tags. This full-chain view must be available in a single analysis session.
- **FR-AN-05 – Read-Only Operation:** All system operations must be read-only with respect to Azure DevOps. The system must not create, modify, or delete any artifact in Azure DevOps under any circumstances.

3.8 Non-Functional Requirements

Non-functional requirements define the quality attributes the system must meet to be suitable for use in the Forvia industrial environment. They constrain how the functional requirements are delivered, with attention to performance, scalability, reliability, security, and maintainability.

3.9 Performance Requirements

Performance is a core concern because the system is meant to replace a slow manual process. If the automated version is not clearly faster, adoption will be limited even if the functional behavior is correct.

- **NFR-PE-01 – Analysis Response Time:** For single work item trace operations involving a dependency graph of typical size, up to thirty related work items across two or three hierarchy levels, the system must return a complete result within a response time that is practical for interactive use.
- **NFR-PE-02 – Batch Processing Efficiency:** For batch analysis sessions involving multiple work item roots, the system must complete processing in a time that grows sub-linearly with the number of roots, using parallel execution and shared caching of common subgraphs.
- **NFR-PE-03 – API Call Minimization:** The system must minimize the number of API calls issued to Azure DevOps for a given analysis session by using batch requests, caching, and deduplication. Unnecessary calls are not acceptable, both for performance and for rate-limit protection.
- **NFR-PE-04 – Responsive User Feedback:** For operations that take more than a few seconds, the system must provide continuous progress feedback through progressive streaming so the interface stays responsive and informative.

3.10 Scalability Requirements

The MES X.0 platform continues to grow. New microservices are added, existing services expand, and the number of development artifacts increases over time. The traceability system must remain usable as that growth continues.

- **NFR-SC-01 – Work Item Volume Scalability:** The system must remain functional and performant as the number of work items in the analyzed Azure DevOps project grows, without requiring changes to the core traversal or resolution logic.
- **NFR-SC-02 – Repository and Service Count Scalability:** The system must handle dependency graphs that span more repositories and microservices over time, while keeping commit and tag resolution efficient through batch processing.

- **NFR-SC-03 – Extensibility for New Artifact Types:** The architecture must allow future additions, such as pipeline run artifacts, automated test results, or release deployment records, without forcing a redesign of the current components or their data contracts.
- **NFR-SC-04 – Configuration-Driven Adaptability:** System behavior that is likely to vary across environments, such as the Azure DevOps organization, project identifiers, or API credentials, must be externalized into configuration so the system can be deployed in different contexts without code changes.

3.11 Reliability Requirements

In an industrial setting, users need to trust the result. If analysis outputs are inconsistent or fragile, people will fall back to manual verification and lose the benefit of the system.

- **NFR-RE-01 – Result Determinism:** Given identical inputs and the same Azure DevOps state, the system must produce identical results across successive analysis sessions. Non-deterministic behavior caused by race conditions, unordered concurrent results, or non-reproducible data access is not acceptable.
- **NFR-RE-02 – Partial Failure Isolation:** If some Azure DevOps API calls fail during analysis because of transient network errors, rate limiting, or partial unavailability, the system must isolate the failure to the affected subset and continue processing the rest of the analysis. Partial failures must be reported clearly and must not silently corrupt the result.
- **NFR-RE-03 – Data Integrity Preservation:** Throughout the traceability pipeline, the system must preserve the integrity of the data it processes. Entity identifiers, relation links, and commit references must remain accurate and unchanged from retrieval through normalization, aggregation, and final output.
- **NFR-RE-04 – Transient Error Recovery:** For external API calls that fail with transient errors such as timeouts or temporary service unavailability, the system must implement a structured retry mechanism before reporting failure to the caller.

3.12 Maintainability Requirements

The system is expected to stay in use and evolve with the MES X.0 platform. Maintainability therefore matters over the long term: the code must remain easy to understand,

change, and extend by engineers who did not build the first version.

- **NFR-MA-01 – Modular Component Structure:** The system must be organized into cohesive modules with clear responsibilities. Each module must be understandable and testable on its own.
- **NFR-MA-02 – Minimal Inter-Component Coupling:** Modules must interact only through documented, typed data contracts. No module may depend on the internal implementation details of another module.
- **NFR-MA-03 – Testability:** The architecture must support unit testing of individual services and utility functions without requiring a live Azure DevOps connection. Business logic must be separable from external API interaction.
- **NFR-MA-04 – Documentation:** All public API surfaces must be documented with automated tooling, specifically Swagger for the backend endpoints, so the reference stays accurate as the implementation evolves.
- **NFR-MA-05 – Naming and Code Consistency:** Naming conventions, code structure, and documentation style must remain consistent across the system to reduce the cognitive effort needed to navigate the codebase.

3.13 Security and Data Integrity Requirements

The system runs in a controlled internal environment, but it still interacts with a corporate Azure DevOps instance and sensitive project data. Security and data integrity therefore need to be explicit requirements.

- **NFR-SE-01 – Input Validation:** All data entering the system through API endpoints, including path parameters, query parameters, and request bodies, must be validated against typed schemas before processing begins. Invalid inputs must be rejected early with clear error responses.
- **NFR-SE-02 – Credential Protection:** Azure DevOps API credentials, specifically Personal Access Tokens, must be managed through environment configuration and must never appear in API responses, log output, or any other externally visible artifact.
- **NFR-SE-03 – Controlled External Access:** All access to Azure DevOps APIs must go through the dedicated integration layer. No other component may directly construct or issue requests to external Azure DevOps endpoints.

- **NFR-SE-04 – Read-Only Access Enforcement:** The Azure DevOps integration must be configured and implemented for read-only use only. Write access to Azure DevOps resources must be technically prevented by the credential configuration, not only by convention.

3.14 Use Case Model

The use case model describes how the actors interact with the system to achieve traceability and analysis goals. It complements the requirement lists with a behavioral view of the system.

3.14.1 Actors

The system has three actors. Two are human users, and one is an external system that provides the data.

Developer and Release Engineer

This is the primary user of the system. Developers use it to understand the impact of a work item on the codebase, while release engineers use it to check release scope and confirm whether changes sit on the correct side of a release boundary.

This actor starts analysis sessions, selects analysis modes, reviews the aggregated results, and uses the Decision Dashboard output to support release, integration, and escalation decisions.

Administrator (DevOps)

This actor manages the operational configuration and availability of the system. They handle environment-level settings such as Azure DevOps organization identifiers, project defaults, API credential management, and deployment configuration. They do not normally use the analytical features directly.

Azure DevOps System

This external actor represents Azure DevOps itself. The system interacts with it programmatically to retrieve work items, pull requests, commits, branches, and tags. It appears in the model because it is the exclusive data source for all retrieval use cases.

3.14.2 Use Case Descriptions

The following use cases describe the system behavior at the level visible to the actors.

Work Item Traceability Use Cases

- **UC-01 – Analyze Single Work Item:** The Developer / Release Engineer provides one work item identifier. The system traverses the full dependency graph rooted at that item, resolves related pull requests and commits, enriches commits with branch and tag context, and returns a complete normalized trace result.
- **UC-02 – Analyze Multiple Work Items:** The Developer / Release Engineer provides several work item identifiers. The system traverses each root, aggregates and deduplicates the results, and returns one consolidated trace result. This supports release scope analysis across a sprint or milestone.
- **UC-03 – Stream Work Item Tasks:** The Developer / Release Engineer starts a batch analysis session. The system streams resolved work item tasks progressively to the client so progress is visible in real time and items can be selected for further analysis before the full dataset is loaded.

Pull Request Traceability Use Cases

- **UC-04 – Retrieve Pull Request Work Items:** The system receives a pull request identifier, either directly from the user or from an upstream component, and returns the list of work items formally linked to that pull request on the Git side, together with the normalized PR summary including the repository name.

Commit Traceability Use Cases

- **UC-05 – Retrieve Commits by Work Item:** Given a work item identifier, the system returns the deduplicated list of commits linked to that work item, resolved through direct work item links and pull request expansion.
- **UC-06 – Retrieve Commits by Pull Request:** Given a pull request identifier, the system returns the list of commits merged as part of that pull request.
- **UC-07 – List Repository Branches and Tags:** Given a repository identifier, the system returns the available branch and tag references needed to interpret commit positions.

- **UC-08 – Resolve Commit Tag Context:** Given a repository and a commit reference, the system determines the tags associated with that commit, specifically the most recent tag reachable from the commit on the relevant branch.
- **UC-09 – Batch Resolve Commit Tag Context:** Given a repository and multiple commit references, the system resolves the tag context of all specified commits in one operation to reduce API usage on large graphs.

Aggregation and Decision Use Cases

- **UC-10 – Generate Dependency View:** After a work item trace is completed, the system aggregates all collected data, including work items, pull requests, commits, and tag context, into a single decision-ready dependency view that classifies work items by completion state and repositories by release-relative commit position.
- **UC-11 – Configure System:** The Administrator updates environment-level configuration, including Azure DevOps connection settings, API credential management, and deployment parameters.

3.14.3 Use Case Diagram

Figure 3.1 presents the complete use case diagram of the system, showing the relationship between the three actors and the eleven use cases.

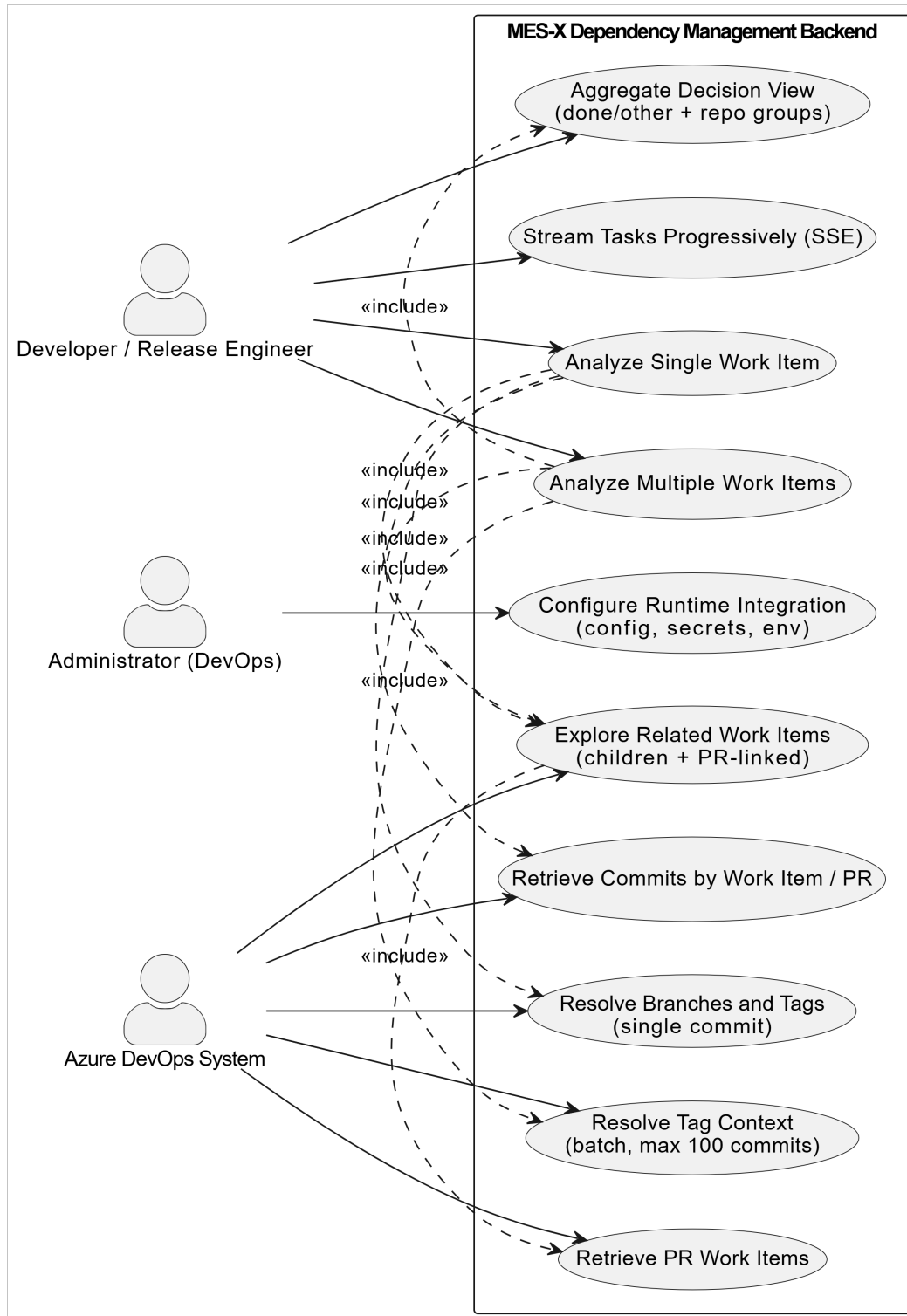


Figure 3.1. Use Case Diagram of the MES-X Dependency Management System.

3.14.4 Scope Boundaries

Two scope boundaries need to be stated clearly.

First, as established by **FR-AN-05**, all use cases are read-only. The system does not create, modify, or delete any artifact in Azure DevOps. It only retrieves and analyzes existing information, so Azure DevOps appears in the model only as a data provider.

Second, notification and alerting capabilities, although mentioned in some internal notes as possible future extensions, are outside the scope of the current system. This specification covers only the traceability and analysis capabilities described above.

3.15 Requirements Traceability Summary

Table 3.1 summarizes how the functional requirement groups map to the system components in the proposed architecture. This mapping creates the link between specification and design and makes it clear which part of the system is responsible for each requirement set.

Table 3.1. Functional Requirements to System Component Traceability Matrix

Requirement Group	Requirement IDs	Addressing Component
Work Item Traceability	FR-WI-01 to FR-WI-06	Work Item Management Component
Pull Request Traceability	FR-PR-01 to FR-PR-04	Pull Request Resolution Component
Commit Traceability	FR-CO-01 to FR-CO-06	Commit Aggregation Component
Data Aggregation	FR-AG-01 to FR-AG-05	Decision Aggregation Component
Analysis and Reporting	FR-AN-01 to FR-AN-05	All components, Trace UI

3.16 Conclusion

This chapter defined the requirements for the MES-X Dependency Management system in a structured and traceable way. The functional requirements cover the five main capability areas of the solution: work item traceability, pull request resolution, commit enrichment, data aggregation, and analytical reporting. The non-functional requirements

describe the quality constraints in the areas of performance, scalability, reliability, maintainability, and security. The use case model turns those requirements into eleven observable actor-system interactions.

Taken together, these requirements establish the reference framework for the rest of the report. They explain not only what the system should do, but also the level of performance, reliability, and maintainability expected in the industrial environment in which it will be used.

The next chapter presents the proposed solution and explains how the chosen architecture addresses these requirements through a layered design and a specialized traceability pipeline.

Chapter 4

Proposed Solution

4.1 Introduction

The previous chapter showed that Azure DevOps stores the necessary traceability data, but does not present it in a form that is easy to analyze end to end. Teams therefore have to move manually between work items, pull requests, commits, branches, and tags to reconstruct the dependency chain of a feature. That approach is workable on a small scale, but it becomes costly and unreliable in a large microservices environment.

The proposed solution addresses this gap through a **centralized dependency traceability system** designed for Azure DevOps-based workflows. Rather than replacing the existing platform, it adds an analytical layer on top of it: the system retrieves data from Azure DevOps APIs, rebuilds dependency graphs, enriches the raw artifacts with additional context, and exposes the result through a dedicated interface.

The system is built around three guiding ambitions:

- **Completeness:** Every artifact in the dependency chain — from epics and features down to individual commits and Git tags — must be reachable and traceable within a single analysis session.
- **Consistency:** All data must be normalized before processing so that results are stable and predictable regardless of the structural variations present in Azure DevOps API responses.
- **Actionability:** The final output must not be a raw data dump. It must be organized into structured, decision-ready views that directly support release readiness assessment, impact analysis, and dependency validation.

The following sections present the global architecture of the solution, the design

principles that govern its construction, the four core components that form the traceability pipeline, the end-to-end data flow connecting them, and the concrete advantages the system delivers over the current state of practice.

4.2 Global System Architecture

The system adopts a **three-layer architecture** that cleanly separates the concerns of user interaction, business orchestration, and external data access. This separation is not merely a structural convention: it is a deliberate design decision that ensures each layer can be developed, tested, and evolved independently without creating cascading changes across the system.

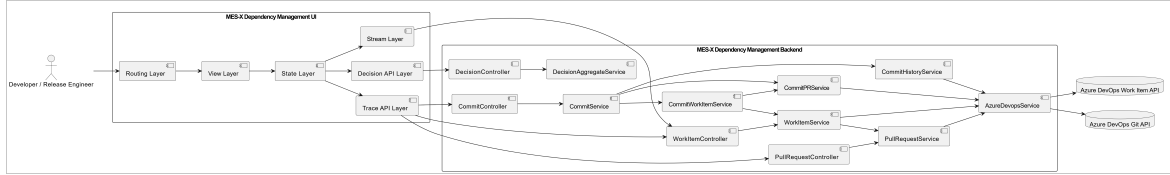


Figure 4.1. Global three-layer architecture of the proposed solution.

4.2.1 Presentation Layer — Trace UI

The Presentation Layer is the entry point for end users. It is implemented as a dedicated web interface — referred to throughout this document as the *Trace UI* — that allows users to initiate dependency analysis sessions, monitor their progress in real time, and explore the resulting dependency graphs and decision outputs.

The Trace UI is not a passive display surface. It actively participates in the orchestration of the analysis by:

- accepting user-defined analysis parameters (work item identifiers, project context, analysis mode);
- managing the lifecycle of asynchronous and streamed backend interactions;
- assembling and normalizing backend responses into a coherent view state;
- presenting structured, layered results across multiple specialized views.

The interface supports three distinct analysis modes — single work item trace, batch work item trace, and batch-related work item trace — each tailored to a specific use case and level of analytical depth.

4.2.2 Application Layer — Backend Services

The Application Layer constitutes the analytical core of the system. It is implemented as a modular backend service responsible for orchestrating all traceability logic: dependency graph traversal, data normalization, cross-module enrichment, and decision aggregation.

This layer receives structured requests from the Trace UI, coordinates calls to the Azure DevOps APIs, applies domain-specific processing rules, and returns normalized, enriched results through well-defined API contracts. It acts as the intermediary that isolates the Presentation Layer from the complexity and variability of the underlying data source.

The Application Layer is internally composed of four specialized components — each described in detail in Section 4.4 — that together form a sequential traceability pipeline.

4.2.3 Integration Layer — Azure DevOps REST APIs

The Integration Layer represents the external data source of the system. Azure DevOps REST APIs serve as the single source of truth for all project artifacts: work items and their hierarchical relations, pull request metadata and linked work items, commit histories, repository branch structures, and Git tag references.

The system interacts with two distinct Azure DevOps API surfaces:

- **Work Item Tracking API:** used to retrieve work items, resolve parent/child and linked-item relationships, and query work item states and metadata.
- **Git API:** used to retrieve pull request details, commit histories, branch references, and tag context for specific commits and repositories.

All interactions with these APIs are encapsulated within a dedicated integration utility layer that handles authentication, request formatting, response normalization, and error management. This encapsulation ensures that the rest of the system never directly depends on the raw Azure DevOps response structures, protecting the application against API changes and simplifying future migrations.

4.2.4 Interaction Between Layers

The three layers interact through a strict unidirectional flow. The Presentation Layer communicates only with the Application Layer, while the Application Layer handles all

interactions with Azure DevOps through the Integration Layer. The frontend never accesses Azure DevOps directly.

This separation keeps responsibilities clear. Changes in external APIs remain isolated in the integration utilities, business rules stay concentrated in the backend, and each layer can be tested independently with controlled inputs and outputs.

4.3 Design Principles

The architecture of the proposed solution is governed by four fundamental design principles. These principles were not chosen arbitrarily: each one addresses a specific category of risk or complexity identified during the problem analysis phase, and each one shapes concrete decisions visible throughout the system design.

4.3.1 Modularity

The system is decomposed into four focused, independent modules: Work Item Management, Pull Request Resolution, Commit Aggregation, and Decision Aggregation. Each module encapsulates a coherent set of responsibilities and exposes its behavior exclusively through a well-defined API surface. No module contains logic that properly belongs to another.

This modularity serves several practical purposes. First, it reduces the cognitive complexity of each individual component, making the system easier to understand and maintain. Second, it limits the impact of changes: a modification to the commit resolution strategy, for example, requires changes only in the Commit Aggregation module and does not cascade into the Work Item or Decision modules. Third, it improves testability by allowing each module to be exercised in isolation with controlled inputs.

4.3.2 Separation of Concerns

Modules interact only through well-defined data contracts — specifically, through typed Data Transfer Objects (DTOs) that specify the exact shape of data crossing module boundaries. No module has direct knowledge of another module’s internal implementation, internal data structures, or external API dependencies.

This principle is particularly important in a system that depends on external APIs with variable response structures. By ensuring that each module only operates on

normalized, contract-conformant data, the system avoids a class of bugs that would otherwise arise from implicit assumptions about data shape. It also ensures that each module’s implementation can be refactored or replaced without affecting its consumers, as long as the contract remains stable.

4.3.3 Data Normalization

All data received from Azure DevOps APIs is immediately transformed into internal DTOs before being used in any processing logic. This normalization step occurs at the boundary of the Integration Layer and is mandatory for every response, regardless of the consuming module.

The purpose of this principle is to create a stable, homogeneous internal data model that all system components can rely on. Azure DevOps API responses are often heterogeneous: the same conceptual entity — such as a pull request link embedded in a work item relation — may appear in different structural forms depending on the API version, the repository configuration, or the type of relation. Without normalization at the entry point, this heterogeneity would propagate throughout the system and require every consuming component to handle structural variability defensively.

Data normalization therefore acts as a *complexity firewall*: it absorbs the variability of the external world at the boundary and exposes a clean, consistent model to the internal domain.

4.3.4 Batch-Oriented Processing

To support performance and scalability, the system favors batch API interactions over individual, item-by-item calls whenever the data access pattern permits it. Work item retrieval, commit resolution, and tag context lookup are all designed around group operations: instead of issuing one API call per entity, the system collects identifiers, groups them by natural boundaries (such as repository or project), and issues a single grouped request.

This design principle is a direct response to two concrete challenges. First, Azure DevOps APIs enforce rate limits that can be triggered by high volumes of sequential individual requests during a large-scale analysis session. Second, individual call patterns introduce significant latency through cumulative network round-trip overhead, particularly when resolving tag context for dozens of commits across multiple repositories.

By adopting a batch-oriented processing model throughout the pipeline, the system

reduces both the total number of API calls and the total elapsed time of an analysis session, making it viable for large projects and portfolios.

4.4 Core Components

The Application Layer is internally organized into four specialized components that together form a complete, sequential traceability pipeline. Each component is responsible for a distinct stage of the analysis: from initial dependency graph construction, through pull request and commit resolution, to final decision aggregation. The components are designed to be composable — the output of each stage becomes the input of the next — while remaining independently testable and evolvable.

4.4.1 Work Item Management Component

Role and Responsibility

The Work Item Management Component is the entry point of the traceability pipeline. Its central function is to transform one or more work item identifiers — provided by the user — into a fully expanded dependency graph that represents all entities reachable from those roots within the Azure DevOps project.

This component is foundational: the quality and completeness of the entire analysis pipeline depends on its ability to reconstruct a reliable dependency graph from the heterogeneous and multi-level relation structures present in Azure DevOps work item data.

Dependency Graph Construction

Work items in Azure DevOps exist within a hierarchical structure — typically organized across three or four levels such as Epics, Features, User Stories, and Tasks — and also carry non-hierarchical links to other artifacts such as pull requests, related items, and predecessor/successor dependencies.

The Work Item Management Component reconstructs this graph through a structured traversal algorithm that proceeds as follows:

1. The traversal begins at the user-specified root work item or set of root items.
2. For each work item, the component retrieves its full relation metadata from the Work Item Tracking API, identifying child links, parent links, related-item links,

and pull request references.

3. Child and related items are queued for processing in the next traversal level. A visited-state registry prevents cycles and avoids reprocessing already encountered nodes.
4. Items are retrieved in batches rather than individually to minimize API round trips.
5. The traversal continues until no unvisited nodes remain in the queue.

The result of this traversal is a normalized graph of work items, together with a deduplicated list of pull request identifiers extracted from the relation links encountered during traversal.

In-Memory Caching Strategy

To further reduce redundant API calls within a single analysis session, the component maintains a request-scoped in-memory cache. Work items already retrieved during a traversal are stored in this cache and returned immediately on subsequent access, without issuing a new network request. This is particularly valuable in scenarios where multiple branches of the dependency graph converge on the same shared work items.

Streaming Support for Batch Analysis

For large-scale batch analysis sessions — where the user selects dozens of root work items for simultaneous processing — the component exposes a streaming endpoint that delivers task batches progressively via the Server-Sent Events (SSE) protocol. This mechanism allows the Trace UI to display incremental progress and intermediate results as they become available, rather than blocking until the entire analysis completes.

Each SSE message carries a structured payload describing the current batch of resolved tasks, the batch sequence number, and the cumulative count of items processed. A terminal message signals the completion of the stream.

Exposed API Surface

- `GET /workitem/:id` — Retrieves a single work item with its normalized relation metadata.

- `GET /workitem/:id/related` — Traverses the full dependency graph rooted at the specified work item and returns the complete normalized result, including all reachable work items, pull request summaries, and commit references.
- `GET /workitem/related` — Accepts multiple root identifiers in a single request and returns one aggregated, deduplicated response covering all reachable items across all roots.
- `GET /workitem/tasks/stream` — Streams task batches progressively over SSE for large-scale batch analysis sessions.

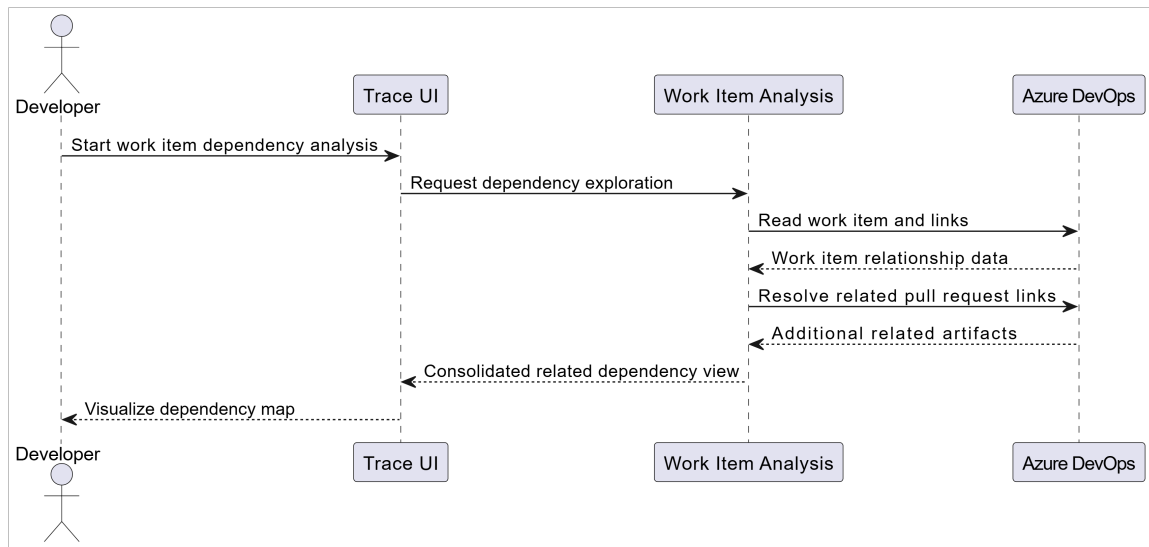


Figure 4.2. Sequence diagram — Work Item Management Component.

4.4.2 Pull Request Resolution Component

Role and Responsibility

The Pull Request Resolution Component bridges the work tracking domain and the source control domain. Its function is to receive the pull request identifiers extracted by the Work Item Management Component and resolve them into enriched, normalized structures that carry both pull request metadata and the identities of the repositories involved.

This bridging role is architecturally significant. In Azure DevOps, the connection between a work item and the code changes that implement it passes through pull requests: a work item links to a pull request, which links to one or more commits in a specific repository. Without explicit pull request resolution, the traceability chain would break at the boundary between the work tracking system and the Git system.

Pull Request Enrichment Process

For each pull request identifier received, the component performs the following operations:

1. Query the Azure DevOps Git API for the full pull request metadata, including title, status, source branch, target branch, and the identifier of the repository in which the PR was created.
2. Extract the list of work items formally linked to the pull request on the Git side — which may differ from the work items that link to the PR on the work tracking side.
3. Construct a normalized `PullRequestSummaryDto` that exposes the PR identifier, the repository name, and the list of associated work item references in a stable, contract-conformant form.

Relationship to Other Components

The Pull Request Resolution Component serves two consumer roles within the pipeline. First, it directly enriches the dependency graph produced by the Work Item Management Component by adding repository-level context to the PR references identified during traversal. Second, it acts as a prerequisite for the Commit Aggregation Component: resolving commits requires knowing which repositories are involved, and this information is produced by pull request resolution.

Exposed API Surface

- `GET /pullrequests/:prId/workitems` — Resolves a pull request by its identifier and returns the normalized PR summary together with the work items formally linked to it on the Git side.

4.4.3 Commit Aggregation Component

Role and Responsibility

The Commit Aggregation Component delivers the finest level of traceability in the system: it connects work items and pull requests to the actual code changes that implement them, and it enriches those code changes with the repository context — branches and tags — needed to determine their position relative to software releases.

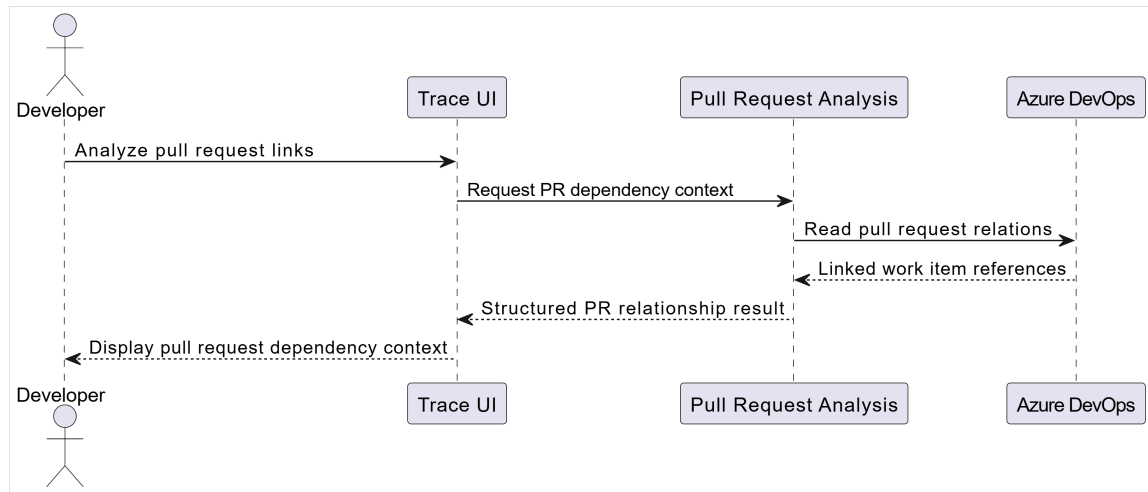


Figure 4.3. Sequence diagram — Pull Request Resolution Component.

This component answers a critical question that no other layer of the system addresses: for a given work item or pull request, which commits exist in which repositories, and what is their release context?

Commit Resolution Strategy

The component resolves commits through two complementary mechanisms:

- **Commit resolution via pull request:** The Azure DevOps Git API provides the list of commits that were merged as part of a specific pull request. This is the primary and most reliable mechanism for establishing the work-item-to-commit link when a pull request is present.
- **Commit resolution via work item:** For work items that carry direct commit references in their relation metadata, the component resolves those references directly through the Work Item Tracking API. This mechanism handles scenarios where commits are linked without a formal pull request.

In both cases, the resolved commits are deduplicated by commit identifier before being passed downstream, ensuring that commits referenced through multiple paths — which is common in complex branching strategies — appear only once in the result.

Repository Context Enrichment

Resolving the commit list is not sufficient for release analysis. Each commit must also be placed in its repository context: which branches contain it, and which tags are

associated with it. This context is critical for the Decision Aggregation Component to determine whether a commit falls before or after a specific release tag.

The component retrieves this context through the Azure DevOps Git API, querying branch and tag references for each resolved commit. To avoid the prohibitive cost of issuing one API call per commit, the component applies the batch-oriented processing principle: it groups commit identifiers by repository and resolves tag context in a single batched request per repository group.

The tag resolution strategy uses a *latest-tag branch strategy*: for each commit, the component identifies the most recent tag reachable from the commit on the relevant branch, providing a meaningful release reference even when a commit is not directly tagged.

Internal Service Decomposition

To maintain clarity and testability, the commit resolution logic is distributed across four specialized internal services rather than concentrated in a single large service class:

- **CommitPRService**: resolves commits associated with a specific pull request via the Git API.
- **CommitWorkItemService**: resolves commits directly linked to a specific work item via the Work Item Tracking API.
- **CommitHistoryService**: queries commit history, branch references, and tag references for specific commits and repositories.
- **Utility services**: provide tag selection logic, batch reference resolution, and commit deduplication shared across the other services.

Exposed API Surface

- **GET /commits/workitems/:workItemId** — Returns the deduplicated list of commits linked to the specified work item, resolved through both direct links and PR expansion.
- **GET /commits/pullrequests/:prId** — Returns the commits associated with the specified pull request.
- **GET /commits/branches** — Lists branch and tag names for a specified repository.
- **GET /commits/tags** — Resolves branch and tag references associated with a specific commit hash.

- `GET /commits/tags/batch` — Batch-resolves tag context for multiple commits across one or more repositories in a single request.

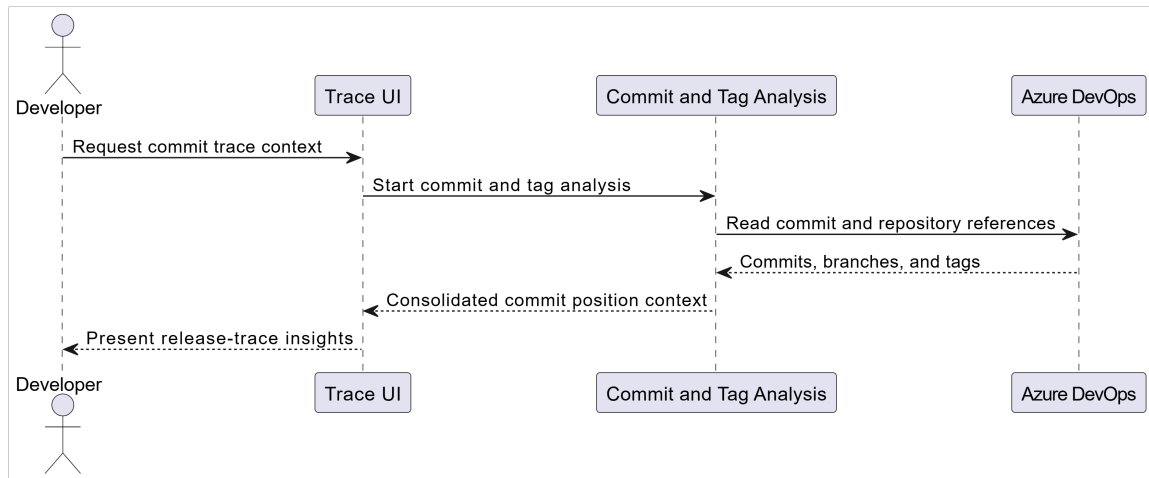


Figure 4.4. Sequence diagram — Commit Aggregation Component.

4.4.4 Decision Aggregation Component

Role and Responsibility

The Decision Aggregation Component represents the final analytical stage of the traceability pipeline. Unlike the three preceding components, it does not issue any queries to Azure DevOps. Instead, it operates entirely on the normalized data produced and collected by the upstream components, applying a classification logic that transforms raw trace data into a structured, decision-ready output.

The central question this component addresses is: *given the full set of work items and their associated commits, what is the release readiness status of each repository involved in the dependency graph?*

Classification Logic

The component receives a consolidated trace payload — containing work items with their states, pull request summaries, and per-repository commit lists with tag context — and applies a multi-step classification process:

1. **Work item status partitioning:** work items are divided into two groups based on their current Azure DevOps state — those whose state maps to a *done* condition (e.g., Closed, Resolved, Done) and those whose state indicates remaining work

(e.g., Active, In Progress, New). This partition establishes whether the functional requirements carried by the work items have been fully delivered.

2. **Repository commit classification:** for each repository present in the commit trace, the component examines the tag context of the associated commits and classifies the repository into one of three release-relative buckets:
 - **Before-and-after:** the repository contains commits both before and after the reference tag, indicating that changes straddle a release boundary.
 - **Only-before:** all commits in the repository predate the reference tag, indicating that the changes were fully included in the referenced release.
 - **Only-after:** all commits in the repository postdate the reference tag, indicating that the changes are part of a future release scope.
3. **Pull request relevance marking:** when a commit should be highlighted for pull request integration, the component sets a dedicated `considerForPR` flag in the output. This makes the recommendation explicit without forcing the user to infer it from raw tag data.

Stateless and Deterministic Design

A key design property of this component is that it is entirely stateless and deterministic. Given the same input trace payload, it will always produce the same classification output. This property makes the component straightforward to test, easy to reason about, and safe to invoke multiple times without side effects.

The stateless design also means that the component can be invoked independently of the rest of the pipeline — for example, to recompute a decision view when the user changes the reference tag — without needing to repeat the upstream data collection steps.

Exposed API Surface

- `POST /decision/aggregate` — Accepts a structured trace payload and returns the classified decision output, including the work item status partition, per-repository decision buckets, and commit-level PR relevance flags.

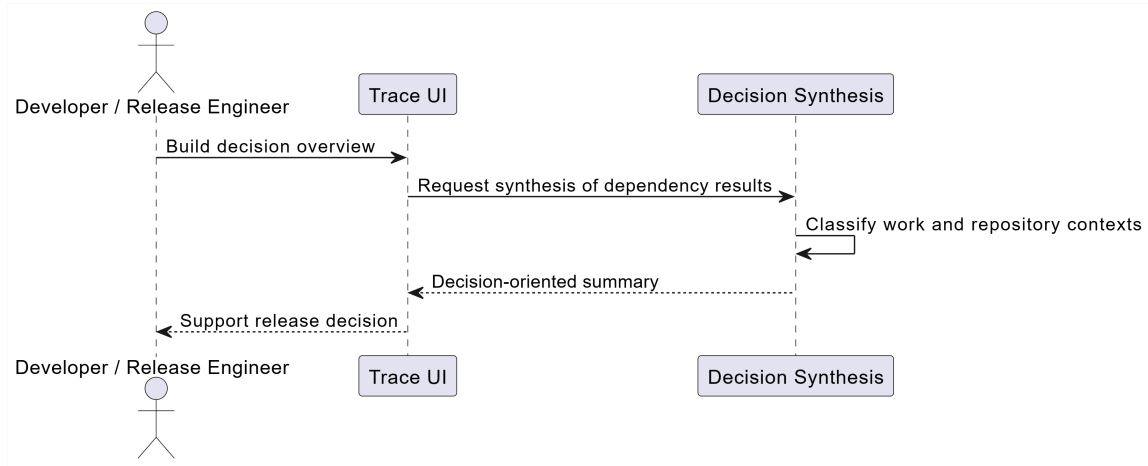


Figure 4.5. Sequence diagram — Decision Aggregation Component.

4.5 Data Contracts and Normalization Model

A critical structural element of the proposed solution is its systematic use of Data Transfer Objects (DTOs) as the exclusive mechanism for data exchange across module boundaries, API surfaces, and layer transitions. This section describes the data contract model and the normalization strategy that underlies it.

4.5.1 Role of Data Transfer Objects

DTOs serve two distinct but complementary roles in the system. At the **input** boundary, they enforce the structure and validity of data entering a module — whether from the Presentation Layer or from an upstream pipeline stage. At the **output** boundary, they define the stable, documented shape of data that a module guarantees to its consumers.

This dual role means that every data transformation in the system is explicit and traceable. When a developer reads the output DTO of a module, they have a complete and accurate description of what that module produces — not an implicit understanding derived from reading implementation code.

4.5.2 Main Data Contract Groups

The system defines five primary groups of data contracts:

- **Work Item Contracts:** `WorkItemDto` represents one normalized work item with its identifier, title, type, state, assignee, and typed relation links through `WorkItemLinkDto`. Single-root traversal results are wrapped in `RelatedWorkItemsDataDto`, while multi-root traversals use `RelatedWorkItemsBatchDataDto`.

- **Pull Request Contracts:** `PullRequestSummaryDto` provides a lightweight normalized representation of a pull request within the traceability context, exposing the PR identifier and the repository name. This DTO is embedded in both work item trace responses and pull request module responses, creating a consistent cross-module reference.
- **Commit and Repository Contracts:** `CommitInfo` carries the commit identifier and repository name as a minimal reference embedded in trace responses. Richer DTOs — `CommitsByWorkItemResponseDto`, `CommitsByPRResponseDto`, `RepoReferencesResponseDto` — provide full commit lists with branch and tag context for consuming components.
- **Streaming Event Contracts:** SSE messages carry structured payloads defined by dedicated event DTOs. Batch progress messages expose the batch content, the batch sequence number, and the cumulative count of resolved items. Completion and error messages follow distinct schemas to allow the consumer to unambiguously identify the event type.
- **Decision Contracts:** the input payload of the final classification stage is described by `DecisionAggregateInputDto`, and the output is described by `DecisionAggregateOutputDto`. Together, they define the work item status groups, repository-level decision buckets, and commit-level pull request relevance flags returned by the system.

4.5.3 Normalization at the Integration Boundary

All normalization from raw Azure DevOps API responses to internal DTOs occurs at the Integration Layer boundary, within shared utility functions dedicated to this transformation. These utilities handle the structural parsing of Azure DevOps relation types, the extraction of embedded identifiers (such as pull request IDs encoded in work item relation URLs), and the mapping of Azure DevOps field names to the internal model.

This boundary normalization ensures that no module ever encounters raw Azure DevOps response data in its processing logic. All processing, classification, and enrichment operates on a clean, homogeneous internal model.

4.6 End-to-End Data Flow

The four core components operate together in a clear, sequential pipeline that connects user intent to actionable analytical output. This section traces the complete execution path of an analysis session, illustrating how each component contributes to the progressive enrichment of the dependency graph.

4.6.1 Step 1 — User Initiates the Analysis

The user accesses the Trace UI and selects an analysis mode. In single-trace mode, the user provides a single work item identifier. In batch mode, the user selects multiple work items from a streamed task list. In batch-related mode, the user triggers an aggregated analysis across all selected roots simultaneously.

The UI validates the user input and constructs a well-formed request that includes the work item identifiers, the Azure DevOps project context, and any additional parameters such as the analysis depth or the reference tag for decision classification.

4.6.2 Step 2 — Work Item Graph Construction

The request reaches the backend, where the Work Item Management Component begins the dependency graph traversal. It retrieves the specified root work items from the Azure DevOps Work Item Tracking API and expands the graph level by level, following parent/child and related-item links. Visited states are tracked to prevent cycles. Items are retrieved in batches, and results are cached to avoid redundant calls.

At the end of this stage, the component has produced:

- a complete, normalized list of all work items reachable from the specified roots;
- a deduplicated list of pull request identifiers extracted from the relation metadata of visited work items.

4.6.3 Step 3 — Pull Request Resolution and Repository Identification

The Pull Request Resolution Component receives the list of pull request identifiers produced in the previous stage. For each identifier, it queries the Azure DevOps Git API to retrieve the PR metadata and the list of work items formally linked to the

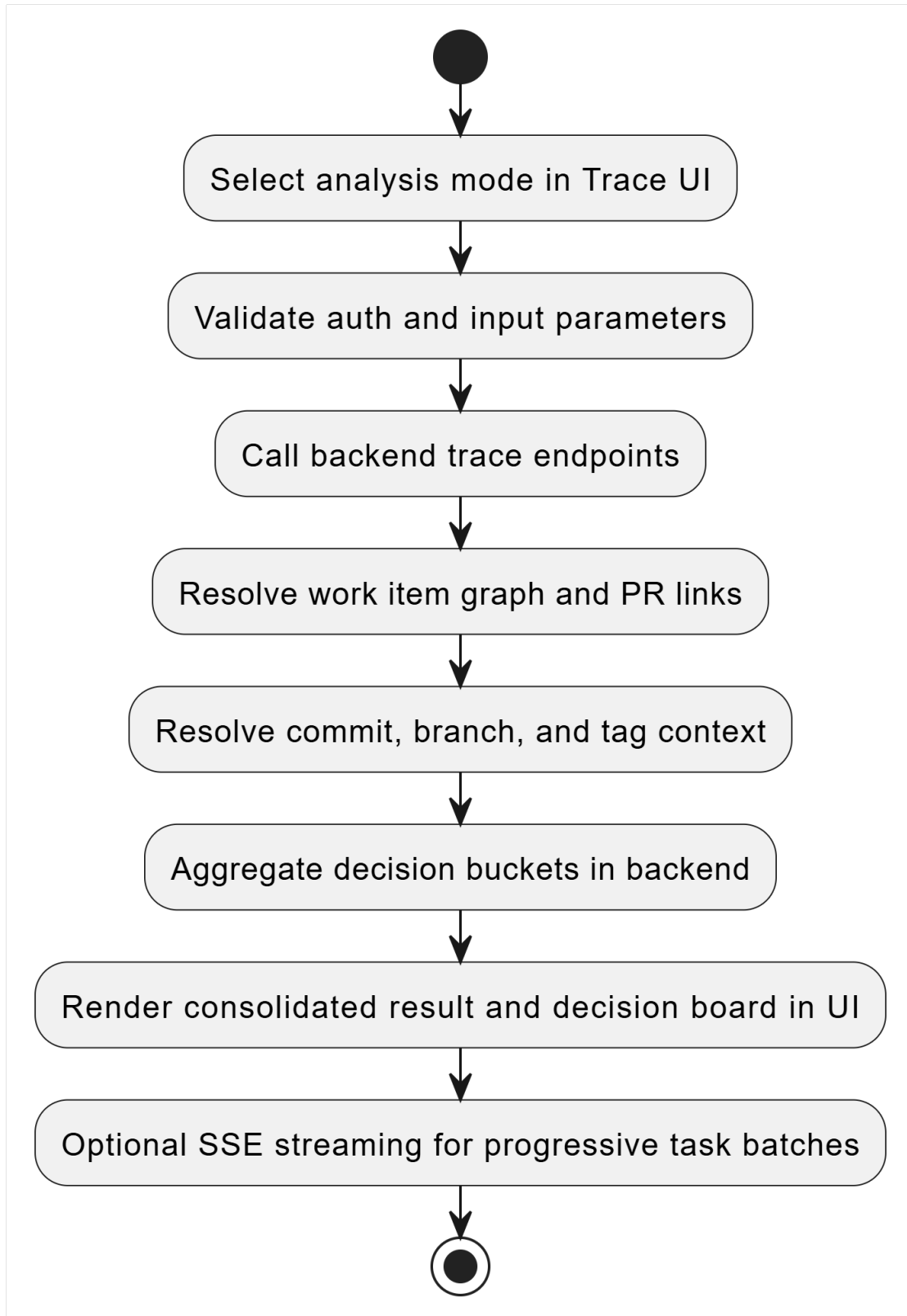


Figure 4.6. End-to-end data flow across the four system components.

PR on the Git side. It constructs a normalized `PullRequestSummaryDto` for each PR, capturing the repository name alongside the PR identifier.

At the end of this stage, the dependency graph has been enriched with:

- normalized pull request summaries associated with the relevant work items;
- the repository identities of all repositories involved in the analysis.

4.6.4 Step 4 — Commit Resolution and Repository Context Enrichment

The Commit Aggregation Component uses the pull request summaries and repository identifiers produced in the previous stage to resolve commits. For each pull request, it queries the Git API to retrieve the list of commits merged as part of that PR. For work items with direct commit links, it resolves those commits independently.

All resolved commits are deduplicated and grouped by repository. The component then performs batch tag resolution: for each repository group, it issues a single API request to resolve the branch and tag context of all relevant commits simultaneously.

At the end of this stage, the dependency graph is complete in the sense that every artifact in the traceability chain — from work item to commit — is represented, normalized, and enriched with release context.

4.6.5 Step 5 — Decision Classification

The Decision Aggregation Component receives the fully enriched trace payload and applies its classification logic. Work items are partitioned by completion state. Repositories are classified into release-relative decision buckets based on the tag context of their associated commits. Commits of particular relevance for pull request integration are flagged accordingly.

The output is a structured decision payload that directly answers the release-readiness question: which repositories are fully integrated, which straddle a release boundary, and which remain outside the current release scope.

4.6.6 Step 6 — Result Delivery and Visualization

The classified decision payload, together with the full trace data, is returned to the Trace UI. The UI assembles the final view state and renders the results across its

specialized view components: the dependency graph view for work item hierarchies, the repository grouping view for commit-level context, and the Decision Dashboard for release classification output.

The user can interact with the results, drill down into specific work items or repositories, and use the Decision Dashboard output to inform release, integration, and escalation decisions.

4.7 User Interface Design and Interaction Model

Although the backend performs the core analysis, the value of the system also depends on how clearly the results are exposed to end users. This section summarizes the three analysis modes supported by the Trace UI and explains the type of user interaction expected in each case.

4.7.1 Single Work Item Trace Mode

In this mode, the user enters one work item identifier and launches a focused analysis. The system traverses the dependency graph rooted at that item, resolves the related pull requests and commits, and presents the result in a structured view. This mode is well suited to targeted impact analysis, for example when verifying the implementation scope of a specific user story.

4.7.2 Batch Work Item Trace Mode

In batch mode, the user receives a progressively loaded list of available work items through the SSE endpoint, selects several of them, and then launches a group analysis. This mode is useful when preparing a release or reviewing a sprint scope, because it reveals all repositories and commits involved in the selected set.

4.7.3 Batch-Related Work Item Trace Mode

This mode performs a parallel dependency analysis across all selected roots and then consolidates the result. The work items can be ranked by the breadth of their dependency footprint, and the Decision Dashboard is displayed to support release-oriented interpretation of the aggregated result.

4.7.4 Observability and Progress Feedback

One important design objective of the Trace UI is to keep users informed during long-running analyses. Thanks to streaming, results appear progressively instead of after one long blocking delay. The interface also exposes the SSE connection state — connecting, open, reconnecting, or closed — so that users can understand what the system is doing at each stage of the analysis.

4.8 Operational Observability and Logging Strategy

Operational observability is treated as a first-class system property, not an afterthought. The system implements a structured logging strategy using the **AppLogger** framework, applied consistently across all backend modules and service layers.

4.8.1 Log Level Strategy

Different log levels are used strategically across the pipeline to maintain a clean operational signal while preserving diagnostic depth when needed:

- **log**: normal operation events — component invocations, traversal progress, batch boundaries, and successful API call completions. These form the standard operational trace of a healthy analysis session.
- **warn**: recoverable anomalies — partial data returns, unexpected API response shapes, rate limit proximity signals, and items skipped due to missing relations. These do not interrupt processing but indicate conditions worth monitoring.
- **error**: failures requiring attention — API call failures after retry exhaustion, data normalization failures, and unresolvable dependency links. These are logged with full context to support root cause analysis.
- **debug** and **verbose**: detailed diagnostic output — individual work item relation resolution, cache hit/miss events, batch composition details, and tag resolution results. These are intended for development and troubleshooting and are suppressed in normal operation.

4.8.2 Observability Value in a Traceability System

Structured logging is especially valuable in a system that depends on external APIs for correctness. When an analysis produces unexpected results — a missing commit,

an incomplete graph, or an incorrect decision classification — structured logs allow the developer or operator to reconstruct the exact sequence of API calls and data transformations that led to the output, without requiring a debugger or a reproduction environment.

The logging strategy also provides an audit trail for analysis sessions, which can be used to verify that a specific analysis was performed correctly and to compare results across successive sessions.

4.9 System Advantages

The proposed solution delivers concrete advantages over the current state of practice, addressing both the functional gaps identified in the problem analysis and broader non-functional requirements of reliability, scalability, and maintainability.

4.9.1 Centralized and Unified Visibility

The most immediate advantage of the solution is the consolidation of scattered dependency information into a single, coherent analytical interface. Before this system, understanding the full dependency chain of a work item required a developer or release engineer to navigate across multiple Azure DevOps views — the work item board, the pull request list, the repository commit history, and the branch/tag management interface — and manually correlate the information found in each.

The proposed system eliminates this manual correlation step. A single analysis session retrieves, normalizes, enriches, and presents the complete dependency chain from work item to commit, in a structured format that supports immediate decision-making.

4.9.2 Improved Release Decision Support

The Decision Aggregation Component translates raw trace data into a structured release classification that directly answers the questions relevant to a release decision: which repositories are fully integrated, which straddle the release boundary, and which changes fall outside the current scope. This structured output reduces the time required to assess release readiness and reduces the risk of releasing incomplete or incorrectly integrated changes.

4.9.3 Scalability for Large Projects

The batch-oriented processing strategy, combined with in-memory caching and SSE-based progressive delivery, makes the system viable for large projects with deep dependency graphs. Analysis sessions involving dozens of root work items, hundreds of related items, and thousands of commits across multiple repositories can be completed efficiently and presented to the user with real-time progress feedback — a use case that is practically infeasible with manual Azure DevOps navigation.

4.9.4 Consistency and Reliability of Results

The systematic normalization of all Azure DevOps API responses at the integration boundary ensures that analysis results are consistent and reliable regardless of the structural variations present in the underlying data. Users can trust that the same query will produce the same result across successive sessions, and that changes in Azure DevOps API response structure will be absorbed at the normalization layer without affecting the rest of the pipeline.

4.9.5 Extensibility for Future Artifact Types

The modular architecture of the Application Layer makes it straightforward to extend the traceability scope beyond the artifact types supported in the current version. New dependency types — such as pipeline run artifacts, automated test results, release deployment records, or cross-project work item links — can be introduced as new modules alongside the existing four, following the same controller/service/DTO pattern and integrating into the pipeline through well-defined data contracts. No modification to the existing modules or the core traversal logic is required.

4.9.6 Shared Reference for Cross-Functional Collaboration

By providing a single, factual, and reproducible view of dependencies, the system creates a shared reference that can be used consistently across roles. Developers, release engineers, technical leads, and project managers can all consult the same analysis output to discuss scope, impact, and readiness — reducing misalignments that arise from each stakeholder navigating the same data through different interfaces and arriving at different conclusions.

4.10 Conclusion

This chapter has presented the proposed solution for structured dependency traceability in Azure DevOps environments. The system is designed around a three-layer architecture, four fundamental design principles, and a sequential pipeline of four specialized components that together transform user-provided work item identifiers into a fully classified, decision-ready dependency view.

Several design decisions stand out as particularly significant. The adoption of a contract-first, DTO-based data model ensures stability and consistency across all module boundaries. The batch-oriented processing strategy makes the system viable at scale without overwhelming external APIs. The integration of SSE-based progressive streaming provides a responsive user experience even during long-running analysis sessions. And the stateless, deterministic design of the Decision Aggregation Component makes the classification logic transparent and easily testable.

Together, these decisions produce a system that is not only functional today but structured for extension and evolution as the traceability requirements of the organization grow.

The following chapter details the technical implementation of this architecture: the technologies, frameworks, and implementation patterns used to realize each component, and the engineering decisions taken during the development process.

Chapter 5

Implementation

5.1 Introduction

This chapter presents the technical implementation of the MES-X dependency traceability system. It explains how the proposed architecture was translated into a working full-stack solution that connects work items, pull requests, commits, branches, and tags into one coherent analytical workflow.

The chapter begins with the technical stack, then describes the development approach, the backend modules, the frontend integration, deployment concerns, and the quality practices used to validate the delivered solution.

5.2 Technology Stack and Architecture

The MES-X implementation is built on a modern full-stack architecture designed to support scalable dependency graph reconstruction and efficient traceability analysis over large datasets.

5.2.1 Web Application Layer (Frontend)

The frontend provides the operational interface used to launch and visualize traceability analyses. It is designed for responsiveness and clarity when handling complex dependency graphs.

Technical specifications:

- **Framework:** Vue 3 with TypeScript

- **UI framework:** Quasar component system
- **Build tool:** Vite
- **Purpose:** trace execution, progressive visualization, and dependency navigation



Figure 5.1. Frontend technology used: Vue 3 for building the user interface

5.2.2 Backend Service Layer

The backend is the core of the MES-X system. It is responsible for executing traceability logic and exposing structured APIs for both single and batch analysis.

Technical specifications:

- **Framework:** NestJS (TypeScript)
- **API styles:** REST for queries and SSE for streaming
- **Role:** orchestration of dependency graph construction

The backend is structured into domain-driven services responsible for processing and enriching traceability data.

5.2.3 Domain Service Layer

This layer contains the core business logic of the system.

- Work item graph traversal and relationship resolution



Figure 5.2. Backend technology used: NestJS for scalable server-side architecture

- Pull request linkage and analysis
- Commit extraction and repository context resolution
- Tag and branch contextual enrichment

Key responsibilities:

- dependency graph construction
- deduplication and cycle prevention
- structured data enrichment for UI consumption

5.2.4 Cloud Infrastructure and DevOps

This layer handles deployment, integration, and operational automation of the system.

- **Source control:** Azure DevOps Git repositories
- **CI/CD pipelines:** automated build, test, and deployment workflows
- **Containerization:** Docker for consistent environment execution
- **External APIs:** Azure DevOps Work Item and Git APIs

This layer ensures that the system remains reproducible, scalable, and deployable across environments with minimal configuration effort.

5.3 Development Methodology

5.3.1 Agile Scrum Execution Model

The development process followed an Agile Scrum methodology adapted to the incremental nature of traceability system development.



Figure 5.3. Azure DevOps Platform

- **Sprint structure:** short iterations focused on incremental feature delivery
- **Backlog management:** user stories prioritized by traceability value and complexity
- **Rituals:** sprint planning, daily stand-ups, reviews, and retrospectives
- **Tracking tool:** Azure DevOps Boards



Figure 5.4. Agile Scrum methodology applied during system development

5.3.2 Implementation Phases

The implementation of MES-X followed a continuous development and integration strategy, where the system was progressively built module by module. Each module went through the full software lifecycle: design, implementation, testing, integration, and deployment.

This incremental approach ensured early validation of core components and reduced integration risks by continuously aligning backend services, frontend interfaces, and DevOps pipelines.

The development was organized into five major phases: Setup, Work Items, Pull Requests, Commits, and Final Release.

Phase 1: Setup and Infrastructure Foundation

The first phase focused on preparing the development environment and the core delivery pipeline of the system.

Main objectives:

- onboarding and environment preparation for developers;
- setup of local development infrastructure and database connections;
- initialization of CI/CD pipelines;
- creation of the frontend project structure and build pipeline.

Key outcomes:

- fully operational development environment;
- automated build pipeline ready for integration;
- initial frontend architecture and design system foundation.

Phase 2: Work Items Module Development

This phase introduced the first core functional module of the system: Work Items management and traceability.

Backend implementation:

- design and implementation of Work Items data model;
- implement service layer logic
- REST API endpoints for querying and filtering;
- validation and schema enforcement.

Frontend implementation:

- Work Items listing and filtering interface;
- creation and editing forms;
- detailed view with status tracking.

Quality and delivery:

- unit testing of backend services;
- end-to-end testing of UI workflows;
- staging deployment and production release.

Phase 3: Pull Requests Module

The second core module focused on linking development activities through Pull Requests, enabling traceability between code changes and work items.

Backend implementation:

- data model design for pull requests;
- storage and filtering logic;
- API endpoints for PR retrieval and aggregation;
- statistical enrichment of PR data.

Frontend implementation:

- PR dashboard and listing view;
- diff viewer for code comparison;
- review and approval workflow interface;
- status indicators and activity timeline.

Delivery process:

- documentation of API and data models;
- unit and integration testing;
- staging validation and production rollout.

Phase 4: Commits and Graph Traceability Engine

This phase represents the core of the MES-X system, introducing deep graph traceability across commits, branches, and repository structure.

Backend implementation:

- commit data model and storage layer;
- graph traversal algorithms for dependency exploration;
- cycle detection mechanisms to avoid infinite traversal;
- performance optimization for large repositories.

Frontend implementation:

- commit history and pagination view;
- interactive graph visualization of dependencies;
- commit details and file-level diff inspection.

Validation and testing:

- unit tests for traversal and API logic;
- UI-based end-to-end graph testing;
- performance validation under large datasets.

Phase 5: System Release and Final Integration

The final phase focused on stabilizing the system and preparing it for full production deployment.

Activities included:

- integration of all modules into a unified system;
- implementation of automated CI/CD validation stages;
- production deployment pipeline configuration;
- post-deployment monitoring and validation.

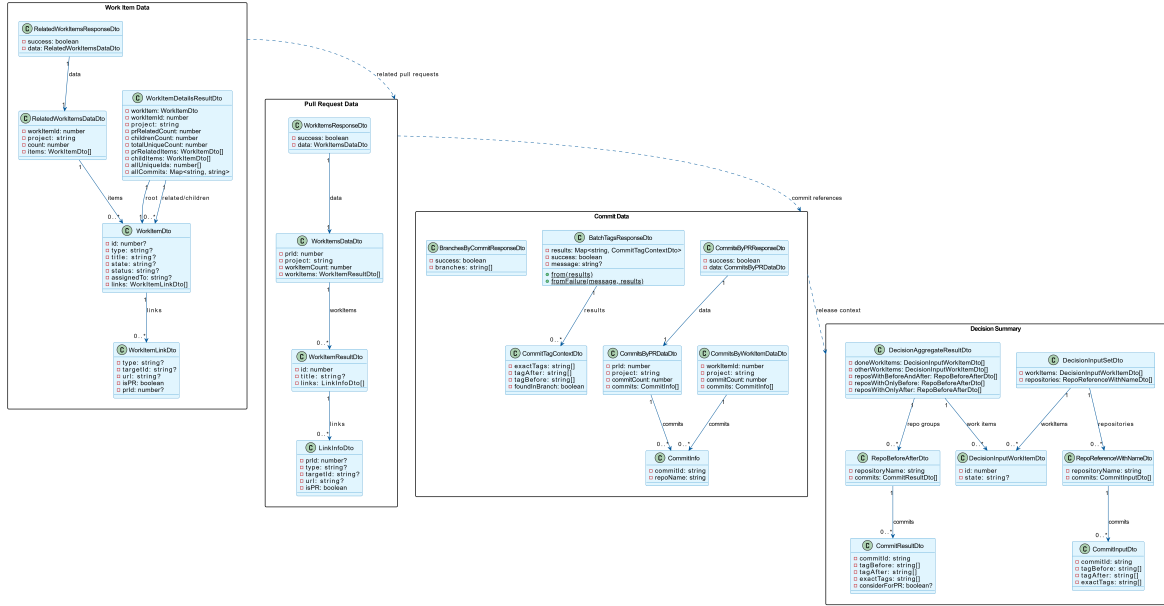


Figure 5.5. Backend module-level class diagram

Final outcome:

- fully integrated MES-X traceability platform;
- stable production-ready deployment pipeline;
- validated end-to-end dependency traceability workflow.

5.3.3 Module-Level Architecture

The backend runtime is organized into domain-focused modules.

- **Work Item module:** endpoints for single-root related traversal, aggregated batch related traversal, and SSE task streaming.
- **Pull Request module:** linked work item retrieval by PR and PR summary enrichment.
- **Commit module:** commit lookup by work item and PR, branch/tag context retrieval, and batch tag resolution.
- **Decision module:** aggregation service for decision-oriented synthesis on top of traceability outputs.

5.3.4 API Contract Design

The backend exposes a set of RESTful endpoints designed to support both single and batch traceability analysis while maintaining a consistent response model.

- **Single related endpoint:** `/workitem/:id/related` Returns a traceability graph centered on a single root work item.
- **Batch related endpoint:** `/workitem/related` Returns an aggregated and deduplicated graph for multiple root items.
- **Tasks stream endpoint:** `/workitem/tasks/stream` Uses Server-Sent Events (SSE) to stream processing results progressively.
- **PR-to-work-items endpoint:** `/pullrequests/:prId/workitems` Retrieves work items linked to a given pull request.
- **Commit tag batch endpoint:** `/commits/tags/batch` Enriches commit references with repository tag metadata.

The DTO strategy emphasizes:

- stable response contracts between backend and service consumers;
- best-effort metadata enrichment (e.g., repository names for visited PRs);
- explicit counters and trace metadata for UI sorting and interpretation.

5.3.5 Implementation Characteristics

- graph-oriented traversal with visited-state safeguards to avoid cycles;
- request-scoped caching and batched retrieval to reduce external API load;
- normalization of external payloads before service-to-service propagation;
- structured logging strategy for diagnosability and operational support.

5.3.6 Sequence and Processing Flows

The backend processing flow is centered on orchestration pipelines where each service enriches the output of the previous stage.

- **Single-root flow:** resolve root work item, traverse related nodes, enrich pull requests and commits, return normalized graph.
- **Batch flow:** aggregate multi-root traversal with deduplication and consolidated counters.
- **Commit tag flow:** resolve tag context in batch by repository grouping and latest-tag strategy.

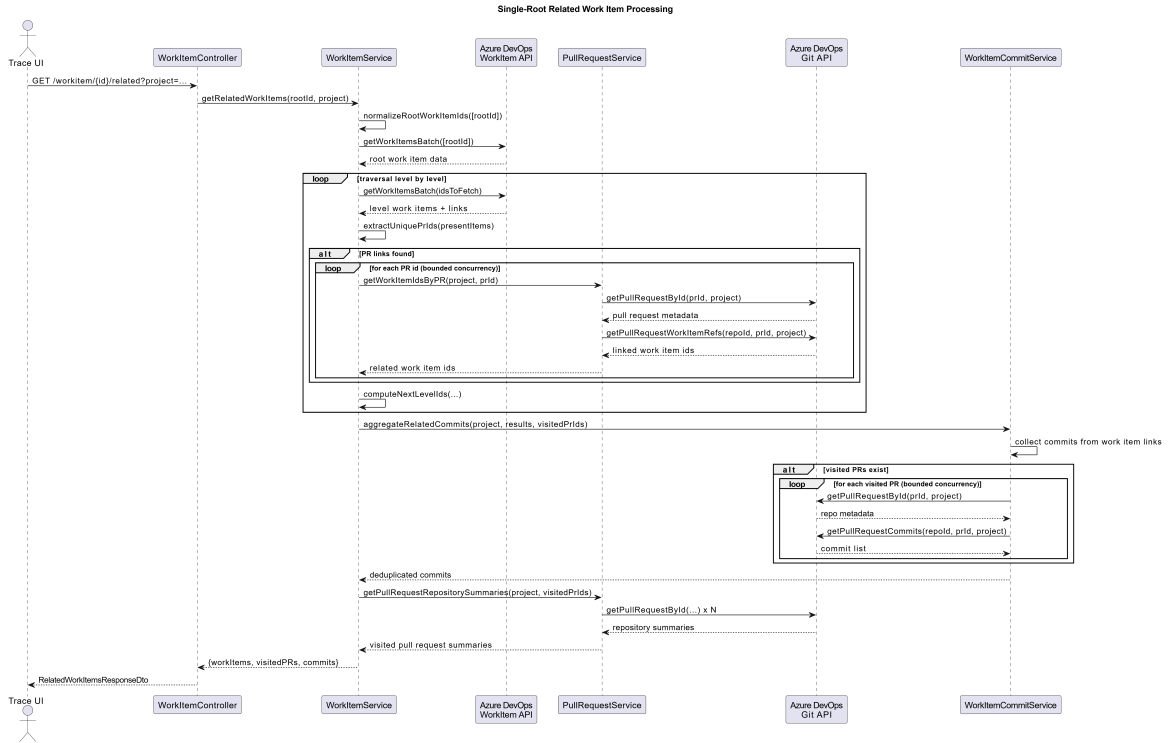


Figure 5.6. Sequence diagram of single-root related-work-item processing

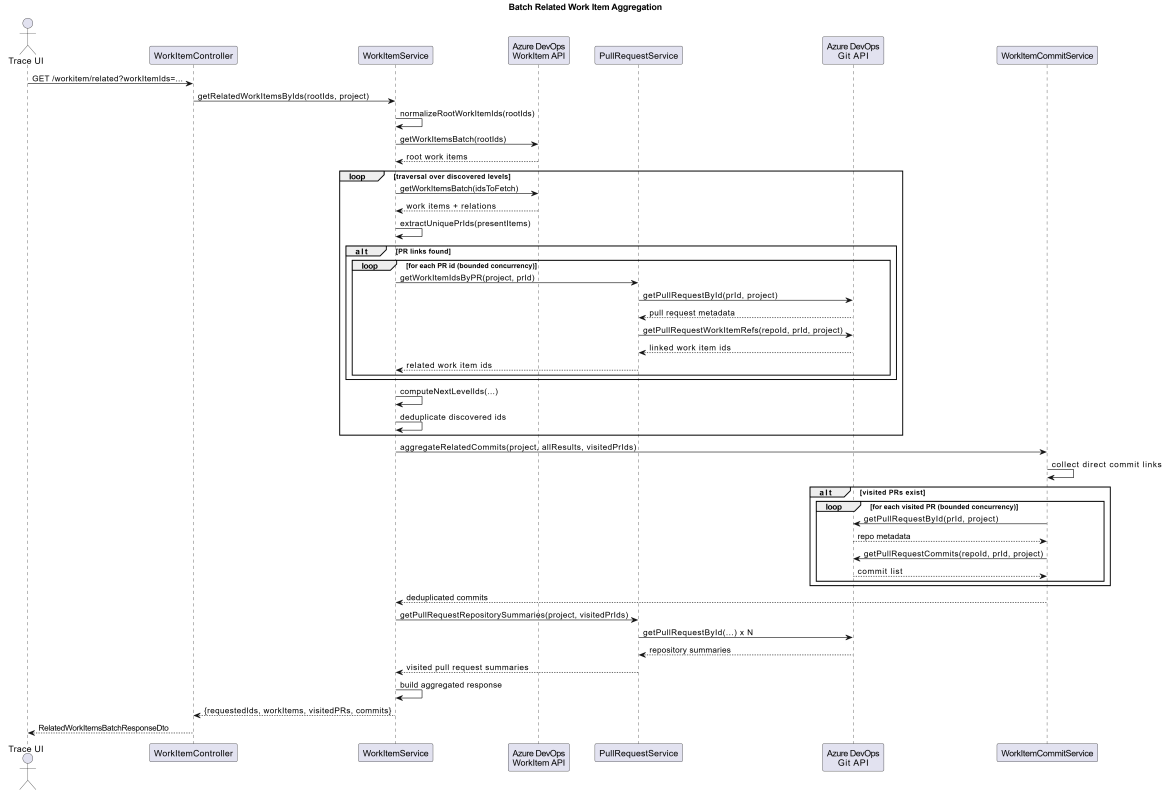


Figure 5.7. Sequence diagram of batch related-work-item aggregation

5.4 UI Implementation

5.4.1 UI Architecture and Main Surfaces

The UI layer provides the operational entry point to the traceability system. It is organized around route-driven views that correspond to the main analysis scenarios and relies on a shared state and API integration layer to coordinate requests, progressive loading, and result presentation.

- **RelatedWorkItemTrace:** single-root analysis screen used to launch the exploration of one work item and inspect its related artifacts.
- **BatchWorkItemTrace:** task selection and loading screen used to progressively collect candidate work items through SSE streaming.
- **BatchRelatedWorkItemTrace:** aggregated result screen used to visualize the consolidated outcome of multi-root analysis with commit and repository context.

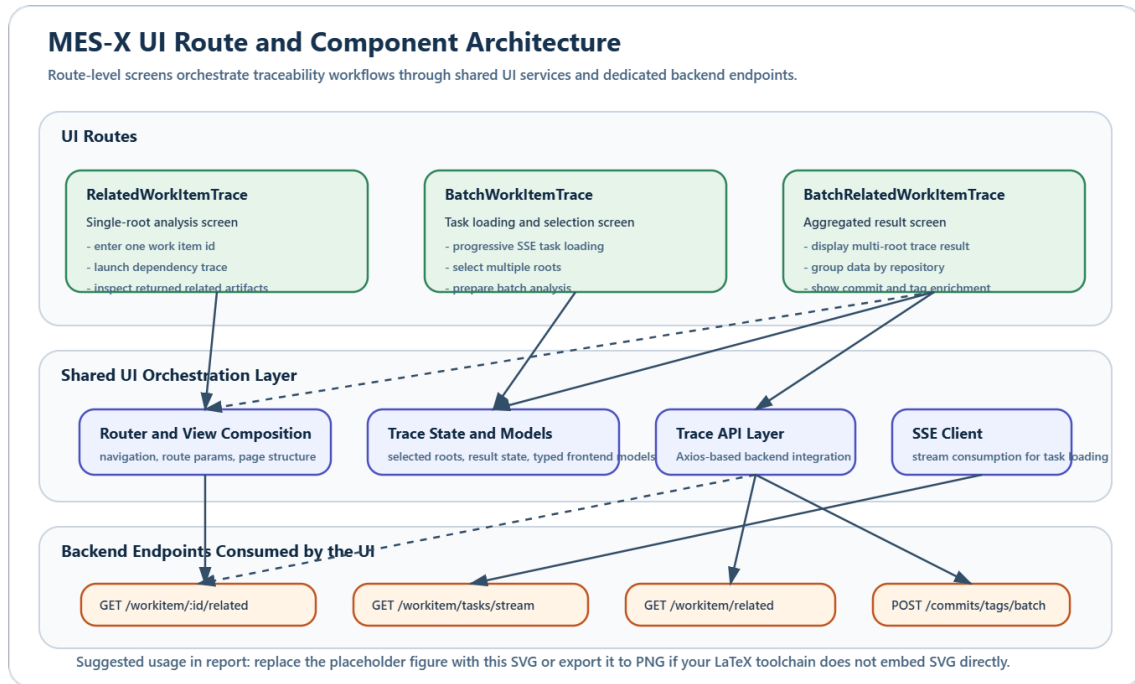


Figure 5.8. UI route/component architecture for traceability workflows

5.4.2 UI Data Flow and Backend Integration

The UI communicates with the backend through a dedicated trace API layer and maps backend contracts to typed frontend models.

- Single analysis flow uses `/workitem/:id/related`.
- Batch task loading consumes `/workitem/tasks/stream` via SSE.
- Batch result enrichment uses `/commits/tags/batch` grouped by repository.
- Authentication failures are handled through controlled redirect behavior when backend returns unauthorized responses.

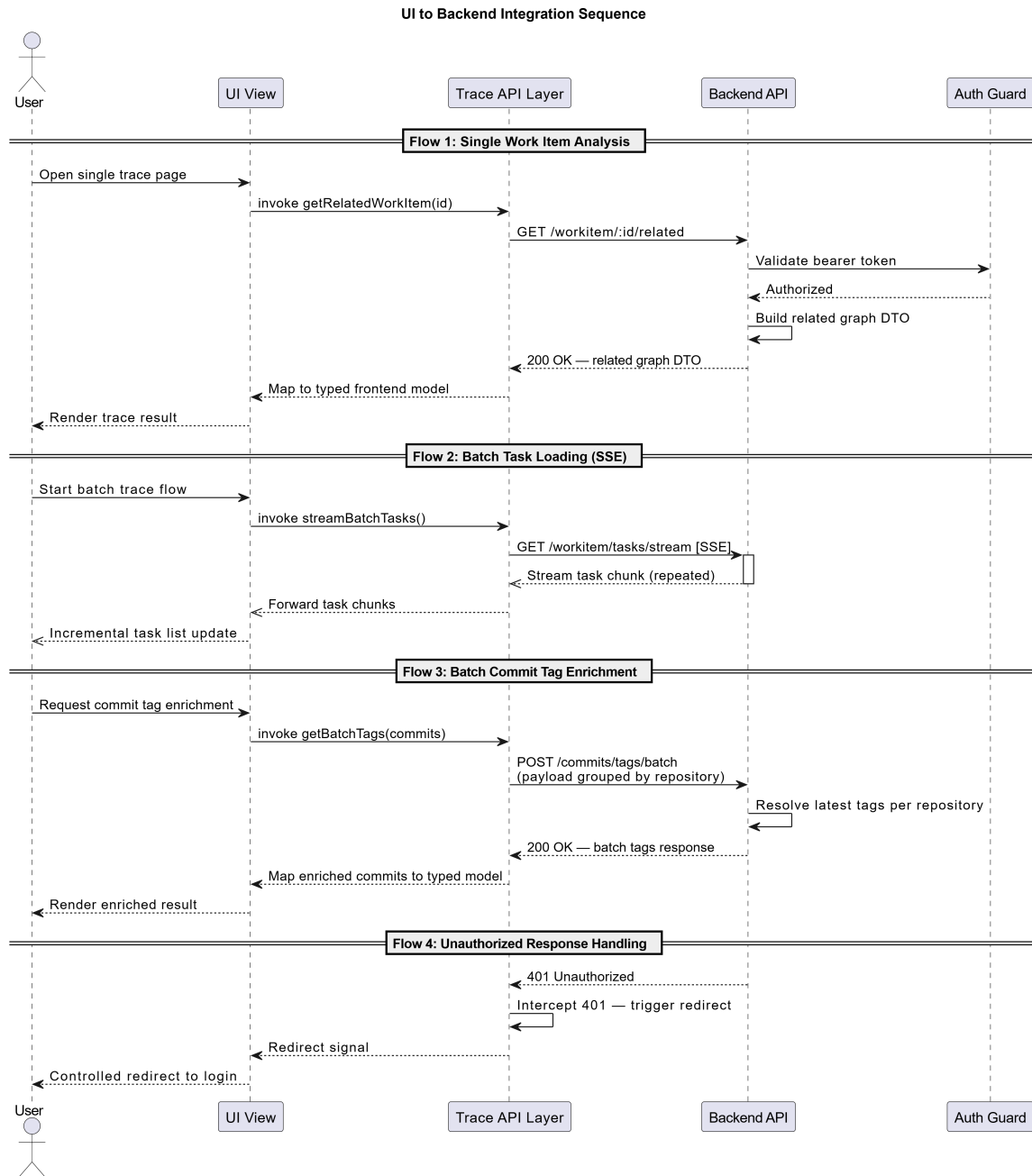


Figure 5.9. Sequence diagram of UI and backend integration in trace workflows

5.4.3 UI Operational Considerations

- progressive loading to keep batch interactions responsive;
- consistent result ordering based on traceability counters;
- fallback rendering when enrichment metadata is partially unavailable.

5.4.4 Screens and UI Walkthrough

Batching Work Items

This screen is used to select multiple work items for batch processing and to start a batched traceability run. Selection is flexible (single, multiple, select-all) and the run progress is shown progressively using Server-Sent Events (SSE). The UI surface provides quick filters and the ability to prioritise candidates for downstream aggregation.

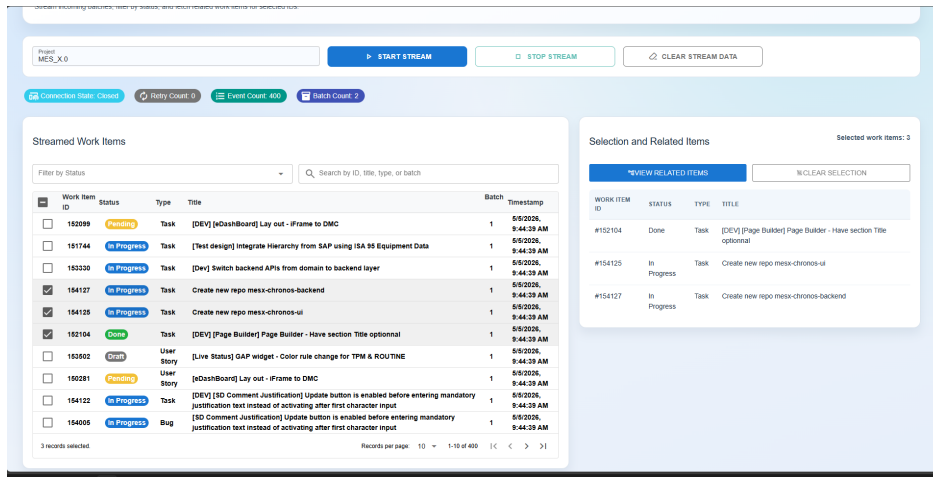


Figure 5.10. Batching Work Items — selection and SSE-driven progress

Get Related

The "Get Related" flow has two complementary screens:

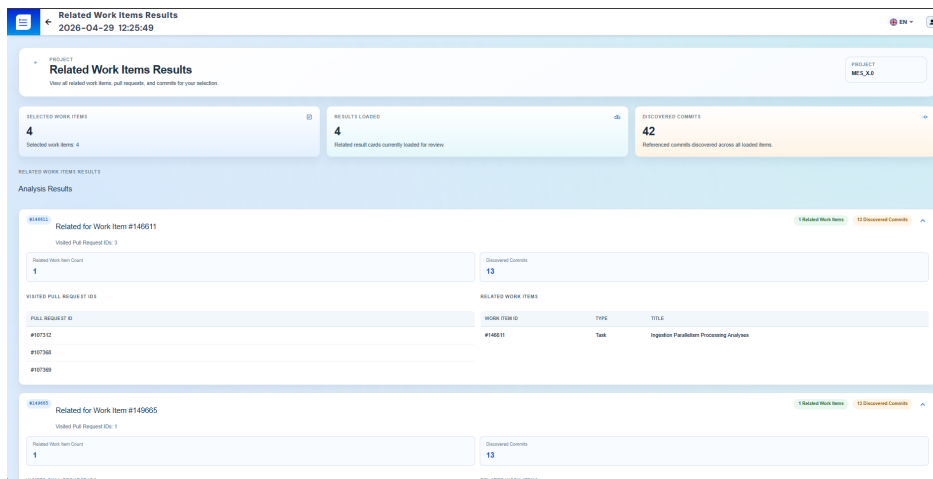
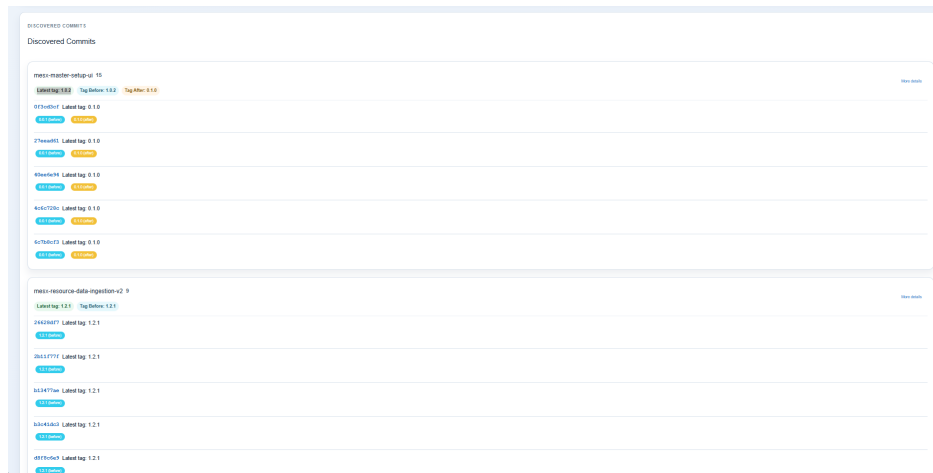


Figure 5.11. Related Work Item — single-root traceability graph view

The Related Work Item view visualises a single root work item and its connected graph (pull requests, commits, branches, tags). It supports expanding nodes, inspecting details and drilling down to the commit or PR level.



The screenshot shows a web interface titled 'Discovered Commits'. It contains two sections of commit data. Each section has a header with a repository name and a 'Tag Before' tag. Below each header is a list of commits, each with a commit ID, a 'Label tag' version, and a 'Tag After' version. Each commit entry has a 'Done' button.

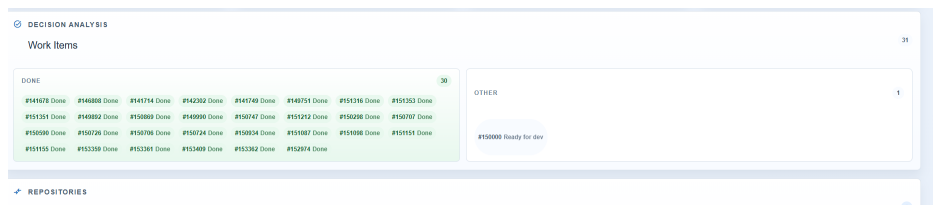
Repository	Tag Before	Commit ID	Label tag	Tag After
mass-resource-mlp-v2-10	Tag Before: 0.1.0	00000000	0.1.0	0.1.0
		01000000	0.1.0	0.1.0
		02000000	0.1.0	0.1.0
		03000000	0.1.0	0.1.0
		04000000	0.1.0	0.1.0
		05000000	0.1.0	0.1.0
		06000000	0.1.0	0.1.0
mass-resource-data-ingestion-v2-9	Tag Before: 1.2.1	07000000	1.2.1	1.2.1
		08000000	1.2.1	1.2.1
		09000000	1.2.1	1.2.1
		10000000	1.2.1	1.2.1
		11000000	1.2.1	1.2.1
		12000000	1.2.1	1.2.1
		13000000	1.2.1	1.2.1

Figure 5.12. *Discovered Commits — commits discovered during traversal*

The Discovered Commits screen lists commits found during traversal together with contextual metadata (branch, tags, and associated work items). This view is useful to verify commit-to-work-item linkage and to inspect tag context before any automated decisions are made.

Decision

The Decision section provides summarised outputs used to guide promotion actions. It contains two key screens:



The screenshot shows a web interface titled 'DECISION ANALYSIS'. It has a 'Work Items' section with two tabs: 'DONE' and 'OTHER'. The 'DONE' tab is active and shows a grid of work items, each with a status 'Done' and a count of 30. The 'OTHER' tab shows a single work item with a status 'Ready for dev' and a count of 1. Below the 'Work Items' section is a 'REPOSITORIES' section.

Work Item ID	Status	Count
#141000	Done	30
#140000	Done	
#141700	Done	
#142000	Done	
#141700	Done	
#140700	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	1
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	
#151300	Done	

Figure 5.13. *Decision — work items grouped by state (Done vs Other)*

The first Decision view groups work items by state (for example, ‘Done’ vs ‘Other’) so operators can quickly identify items that are ready for promotion or require further attention.

The repository/tags screen shows, per repository, the commits and the expected tags (before/after/exact) that the system identified. From here an operator can select a candidate work item and trigger a promotion workflow (for example: move selected work item to production) after validating tag context.

DECISION ANALYSIS		
Repository Conclusions		
mesx-infrastructure-app-proxy	Has evolution commits	
SHOULD BE ON VERSION	1.14.0	CONSIDER MOVING COMMITS TO PR N/A N/A
mesx-proto-dependency-mgt-be		
SHOULD BE ON VERSION	N/A	CONSIDER MOVING COMMITS TO PR 66 commits to move
mesx-proto-dependency-mgt-ui	Has released commits	
SHOULD BE ON VERSION	0.0.1	CONSIDER MOVING COMMITS TO PR 33 commits to move

Figure 5.14. Decision — repository view with expected tags for promotion

5.5 Docker Orchestration and Deployment View

5.5.1 Service Orchestration Model

The backend runtime is packaged as an isolated service with explicit outbound integration boundaries.

- Backend container exposes REST/SSE APIs on the configured runtime port.
- Runtime configuration and secrets are injected through environment variables and externalized config.
- Backend service accesses Azure DevOps Work Item and Git APIs over HTTPS.

5.5.2 Operational Dependencies

- CI/CD pipeline for image build and deployment validation.
- Optional caching layer support through internal cache services.
- Secrets and environment values are injected at runtime.

5.5.3 Deployment and Operations Objectives

- reproducible local and CI execution;
- independent versioning and rollout of backend artifacts;
- reduced deployment risk through clear component ownership.

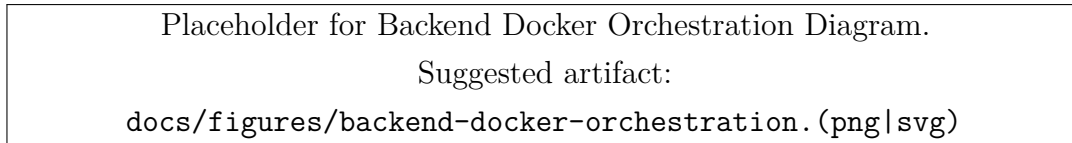


Figure 5.15. *Backend Docker orchestration and runtime communication model*

5.6 Quality, Security, and Operational Considerations

5.6.1 Testing and Validation

Validation was carried out at several levels to ensure that the delivered solution remained consistent with the requirements defined earlier in the report.

- Backend unit tests were used to verify controller and service behavior, especially around work item traversal, commit resolution, and DTO mapping.
- End-to-end tests were used to validate the main API flows exposed by the backend and to confirm that the frontend consumed the contracts correctly.
- Frontend quality controls combined linting, style checks, and manual workflow validation for the main traceability views and the decision dashboard.
- Cross-layer checks were carried out to ensure that streaming behavior, repository grouping, and decision outputs remained coherent from backend to UI.

Quality controls:

- controller-level input validation and typed DTO contracts;
- service-level tests for traversal, tag selection, and normalization;
- regression-focused tests for traversal and data normalization behavior.

Security and runtime controls:

- secure communication for backend inbound and outbound integrations;
- managed external integration access to Azure DevOps;
- structured logging to support incident analysis and diagnostics.

5.7 AI-assisted Development and Review

AI-assisted tools were used throughout the project as a support mechanism for both backend and frontend work. Their role was not to replace engineering decisions, but to accelerate repetitive tasks, improve review coverage, and formalize project-specific guardrails.

The two repositories keep their automation artifacts inside the codebase. On the backend side, the `‘github/agents/‘` directory defines dedicated roles such as architect, implementer, reviewer, and QA agents. On the frontend side, `‘github/agents/‘`, `‘github/instructions/‘`, and `‘github/skills/‘` capture UI guardrails, review rules, and reusable workflows for Vue, Quasar, styling, and quality checks. Keeping these artifacts versioned with the source code made the automation explicit, reviewable, and reproducible.

In practice, these agents were used to assist with code drafting, refactoring, test scaffolding, documentation, and architecture reviews. The instructions helped enforce conventions such as typed contracts, logging discipline, component structure, localization, and style consistency. On the frontend, they supported guardrails for Vue Composition API usage, token-based styling, and UI quality checks. On the backend, they helped maintain consistency in DTO usage, controller-service separation, and review expectations around observability and integration patterns.

All AI-generated suggestions remained subject to human review. Proposed changes were treated as drafts, then validated through linting, SonarQube policies, tests, and direct developer inspection before integration. This approach made AI assistance useful across the full stack while preserving accountability, technical rigor, and architectural coherence.

Conclusion

The implementation delivered a modular traceability platform that combines a graph-oriented backend, an operational frontend, and deployment-ready runtime configuration. The resulting system replaces a manual and fragmented analysis process with a structured workflow that is faster, easier to interpret, and better suited to the needs of release preparation.

By combining layered design, iterative development, quality validation, and careful integration with Azure DevOps, the project established a maintainable foundation for dependency analysis, release decision support, and future extension of the traceability scope.